

https://bioinfmsc8.bio.ed.ac.uk/AY25_BPSM20.html

Quick links:

Click on any 🏠 to get back here!

- [The structure of BioPython](#)
 - [Sequence objects](#)
 - [Codon usage](#)
 - [Reading sequence files](#)
 - [FASTA format](#)
 - [Using `next\(\)`](#)
 - [seq.list of records as a dictionary](#)
 - [GenBank formats](#)
 - [Dealing with features](#)
 - [Feature qualifiers](#)
 - [Getting the sequence for a feature](#)
 - [Searching NCBI databases](#)
 - [Getting and using an NCBI API key](#)
 - [Searching NCBI databases for multiple sequences](#)
-

Exercises

1. How many complete COX1 protein records are there for mammals?
 2. What is their average length (the proteins that is, not the mammals!)?
 3. Write a function that will answer the question for any gene name and any taxonomic group
 4. What else could you sensibly include in the function (in terms of retrieving possibly useful data)?
-
- [important git/GitHub stuff at the end](#)

Please note that BioPython **MUST/SHOULD NOT** be used for the **Optional Python ICA2!**

🏠The structure of BioPython

We have seen some modules that come with Python:

`import re` for regular expressions
`import sys` for system interaction e.g. command line arguments
`import os` and `import shutil` for working with filesystems

Modules are essentially collections of related functions/methods/data types; BioPython is a collection of related modules for working with biological data:

- sequences and annotations
- multiple sequence alignments
- similarity searching
- querying databases
- protein structures
- population genetics
- phylogenetics and trees
- domain motifs
- clustering methods
- machine learning
- graphics
- *etc. etc.*

There is a **huge** on-line resource for using BioPython, available in a tutorial format:
<http://biopython.org/DIST/docs/tutorial/Tutorial.html>

📌 Sequence objects

BioPython has objects to represent sequences which can do common tasks. We can do a lot with very little code!

Remember how we used dictionaries to translate DNA to protein?

Here's a dict that stores a codon usage table:

```
gencode = {
    'ATA': 'I', 'ATC': 'I', 'ATT': 'I', 'ATG': 'M',
    'ACA': 'T', 'ACC': 'T', 'ACG': 'T', 'ACT': 'T',
    'AAC': 'N', 'AAT': 'N', 'AAA': 'K', 'AAG': 'K',
    'AGC': 'S', 'AGT': 'S', 'AGA': 'R', 'AGG': 'R',
    'CTA': 'L', 'CTC': 'L', 'CTG': 'L', 'CTT': 'L',
    'CCA': 'P', 'CCC': 'P', 'CCG': 'P', 'CCT': 'P',
    'CAC': 'H', 'CAT': 'H', 'CAA': 'Q', 'CAG': 'Q',
    'CGA': 'R', 'CGC': 'R', 'CGG': 'R', 'CGT': 'R',
    'GTA': 'V', 'GTC': 'V', 'GTG': 'V', 'GTT': 'V',
    'GCA': 'A', 'GCC': 'A', 'GCG': 'A', 'GCT': 'A',
    'GAC': 'D', 'GAT': 'D', 'GAA': 'E', 'GAG': 'E',
    'GGA': 'G', 'GGC': 'G', 'GGG': 'G', 'GGT': 'G',
    'TCA': 'S', 'TCC': 'S', 'TCG': 'S', 'TCT': 'S',
    'TTC': 'F', 'TTT': 'F', 'TTA': 'L', 'TTG': 'L',
    'TAC': 'Y', 'TAT': 'Y', 'TAA': '_', 'TAG': '_',
    'TGC': 'C', 'TGT': 'C', 'TGA': '_', 'TGG': 'W'}
```

How about doing it the BioPython way?

```
from Bio.Seq import Seq
```

```
my_sequence = Seq("gcgctgattagcggcattgtgattaacggcacccatgaaacgcgctgaaatga").upper()
```

```
my_sequence
```

```
Seq('GCGCTGATTAGCGGCATTGTGATTAACGGCACCCATGAAACGCGCTGAAATGA')
```

```
print(my_sequence)
```

```
GCGCTGATTAGCGGCATTGTGATTAACGGCACCCATGAAACGCGCTGAAATGA
```

```
print(my_sequence.reverse_complement())
```

```
TCATTTTCAGCGCGGTTTCATGGGTGCCGTTAATCACAATGCCGCTAATCAGCGC
```

```
print(my_sequence.translate())
```

```
ALISGIVINGTHETALK*
```

```
print(my_sequence.translate(to_stop=True))
```

```
ALISGIVINGTHETALK
```

Now do the same with the reverse complement

```
print(my_sequence.reverse_complement().translate())
```

```
SFQRGFMGAVNHNAANQR
```

```
print(my_sequence.reverse_complement().translate(to_stop=True))
```

```
SFQRGFMGAVNHNAANQR
```

📌 Not all codon usage tables are the same...

<https://www.ncbi.nlm.nih.gov/Taxonomy/Utils/wprintgc.cgi>

1. The Standard Code
2. The Vertebrate Mitochondrial Code
3. The Yeast Mitochondrial Code
4. The Mold, Protozoan, and Coelenterate Mitochondrial Code and the Mycoplasma/Spiroplasma Code
5. The Invertebrate Mitochondrial Code
6. The Ciliate, Dasycladacean and Hexamita Nuclear Code
7. The Echinoderm and Flatworm Mitochondrial Code
8. The Euplotid Nuclear Code
9. The Bacterial, Archaeal and Plant Plastid Code
10. The Alternative Yeast Nuclear Code
11. The Ascidian Mitochondrial Code
12. The Alternative Flatworm Mitochondrial Code
13. Chlorophycean Mitochondrial Code
14. Trematode Mitochondrial Code
15. Scenedesmus obliquus Mitochondrial Code
16. Thraustochytrium Mitochondrial Code
17. Pterobranchia Mitochondrial Code
18. Candidate Division SR1 and Gracilibacteria Code
19. Pachysolen tannophilus Nuclear Code
20. Karyorelict Nuclear
21. Condyllostoma Nuclear
22. Mesodinium Nuclear
23. Peritrich Nuclear
24. Blastocrithidia Nuclear
25.

Try different tables to see what happens

```
print(my_sequence.translate(table=1))
```

ALISGIVINGTHETALK*

```
print(my_sequence.translate(table=2))
```

ALISGIVINGTHETALK W

```
print(my_sequence.translate(table=3))
```

A T ISGIVINGTHETA T K W

```
print(my_sequence.translate(table=9))
```

ALISGIVINGTHETAL NW

📁 Reading sequence files in FASTA format

BioPython also has methods for reading and parsing common file formats.

```
# Let's use the file called sequences.fasta in the directory /localdisk/data/BPSM/Lecture20/
```

```
# Check the file is what we think it is!
```

```
import os
```

```
os.system("cat /localdisk/data/BPSM/Lecture20/sequences.fasta")
```

```
>ABC123
ATCGTACGATCGATCGATCGCTAGACGTATCG
>DEF456
ACTGATCGACGATCGATCGATCAGACT
>HIJ789
ACTGACACTGTACTGTACATGTG
```

```
# Exit code
```

```
0
```

```
# Read in the file of sequences and translate each of them
```

```
my_fastafile="/localdisk/data/BPSM/Lecture20/sequences.fasta"
```

```
from Bio import SeqIO
records = SeqIO.parse( my_fastafile, "fasta" )
records
```

```
<Bio.SeqIO.FastalO.Fastaliterator object at 0x7f5d624c8ee0>
```

```
# records behaves like a list
```

```
for seq_record in records:
    print( seq_record.id )
    print("\t" + seq_record.seq.translate() )
```

```
ABC123
/usr/lib/python3/dist-packages/Bio/Seq.py:2880: BiopythonWarning:
Partial codon, len(sequence) not a multiple of three.
Explicitly trim the sequence or add trailing N before translation.
This may become an error in future.
    warnings.warn(
        IVRSIDR*TY
DEF456
    TDRRSIDHD
HIJ789
    TDTVLYM

    IVRSIDR*TY
DEF456
    TDRRSIDHD
HIJ789
    TDTVLYM
```



We could use the `next()` method to cycle through the fasta sequences one by one

```
records = SeqIO.parse(my_fastafile, "fasta")
```

`records`

<Bio.SeqIO.FastalO.Fastaliterator object at 0x7f5d5ab2d520>

Use `next()` for the first time

`next(records)`

```
SeqRecord(seq=Seq('ATCGTACGATCGATCGATCGCTAGACGTATCG'), id='ABC123', name='ABC123',  
description='ABC123', dbxrefs=[])
```

Use `next()` for the second time

`next(records)`

```
SeqRecord(seq=Seq('ACTGATCGACGATCGATCGATCAGACT'), id='DEF456', name='DEF456',  
description='DEF456', dbxrefs=[])
```

Use `next()` for the third time

`next(records)`

```
SeqRecord(seq=Seq('ACTGACACTGTACTGTACATGTG'), id='HIJ789', name='HIJ789', description='HIJ789',  
dbxrefs=[])
```

Use `next()` for the fourth time, gives an error as we only had three sequences

`next(records)`

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
    File "/usr/lib/python3/dist-packages/Bio/SeqIO/Interfaces.py", line 85, in __next__  
        return next(self.records)  
StopIteration
```

`records`

<Bio.SeqIO.FastalO.Fastaliterator object at 0x7f5d5ab2d520>

Use `next()` for the fifth time, still gives an error

`next(records)`

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
    File "/usr/lib/python3/dist-packages/Bio/SeqIO/Interfaces.py", line 85, in __next__  
        return next(self.records)  
StopIteration
```

Regenerate `records`

```
records = SeqIO.parse(my_fastafile, "fasta")
```

Use `next()` for the "first" time, get the first record again

`next(records)`

```
SeqRecord(seq=Seq('ATCGTACGATCGATCGATCGCTAGACGTATCG'), id='ABC123', name='ABC123',  
description='ABC123', dbxrefs=[])
```

Other representations are possible too....



```
# We can turn the seq.list of records into a  
# dictionary of Key/Value pairs using SeqIO twice  
# records.id is the key, records.seq are values
```

```
ids_dict = SeqIO.to_dict(SeqIO.parse(my_fastafile, "fasta"))  
ids_dict[ABC123]
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
NameError: name 'ABC123' is not defined
```

```
# Try again, this works as the key is a string!
```

```
ids_dict['ABC123']
```

```
SeqRecord(seq=Seq('ATCGTACGATCGATCGATCGCTAGACGTATCG'), id='ABC123', name='ABC123',  
description='ABC123', dbxrefs=[])
```

```
list(ids_dict.keys())
```

```
['ABC123', 'DEF456', 'HIJ789']
```

```
ids_dict.values()
```

```
dict_values([  
  SeqRecord(seq=Seq('ATCGTACGATCGATCGATCGCTAGACGTATCG'), id='ABC123', name='ABC123', descr  
  dbxrefs=[]), SeqRecord(seq=Seq('ACTGATCGACGATCGATCGATCAGACT'), id='DEF456', name='DEF45  
  description='DEF456', dbxrefs=[]), SeqRecord(seq=Seq('ACTGACACTGTACTGTACATGTG'), id='HIJ  
  description='HIJ789', dbxrefs=[])  
])
```

```
len(ids_dict.items())
```

```
3
```

```
list(ids_dict.values())[0]
```

```
SeqRecord(seq=Seq('ATCGTACGATCGATCGATCGCTAGACGTATCG'), id='ABC123', name='ABC123',  
description='ABC123', dbxrefs=[])
```

```
print(list(ids_dict.values())[0])
```

```
ID: ABC123  
Name: ABC123  
Description: ABC123  
Number of features: 0  
Seq('ATCGTACGATCGATCGATCGCTAGACGTATCG')
```

📖 Reading sequence files in GenBank format

We can do a lot more with GenBank files....

We have a file called `sequence.gb` in `/localdisk/data/BPSM/Lecture20`

```
# Check the file is what we think it should be
```

```
os.system("head -n15 /localdisk/data/BPSM/Lecture20/sequence.gb")
```

```
LOCUS      NC_001861                13747 bp    DNA        circular INV 14-SEP-2017
DEFINITION Onchocerca volvulus strain forest mitochondrion, complete genome.
ACCESSION  NC_001861
VERSION    NC_001861.1
DBLINK     BioProject: PRJNA11839
KEYWORDS   RefSeq.
SOURCE     mitochondrion Onchocerca volvulus (river blindness roundworm)
  ORGANISM Onchocerca volvulus
            Eukaryota; Metazoa; Ecdysozoa; Nematoda; Chromadorea; Rhabditida;
            Spirurina; Spiruromorpha; Filarioidea; Onchocercidae; Onchocerca.
REFERENCE  1 (bases 1 to 13747)
  AUTHORS  Keddie,E.M., Higazi,T. and Unnasch,T.R.
  TITLE    The mitochondrial genome of Onchocerca volvulus: sequence,
            structure and phylogenetic analysis
  JOURNAL  Mol Biochem Parasitol 95 (1), 111-127 (1998)
```

```
# Exit code
```

```
0
```

```
# Now use SeqIO to read it into Python
```

```
my_gbfile = "/localdisk/data/BPSM/Lecture20/sequence.gb"
```

```
GenBankseq = SeqIO.parse(my_gbfile, "genbank")
```

```
GenBankseq
```

```
<Bio.SeqIO.InsdcIO.GenBankIterator object at 0x7f5d5ab2d4f0gt;
```

```
# Need to get at the data inside, so access the next (first) record!
```

```
# Let's keep it as a variable called mito
```

```
mito = next(GenBankseq)
```

```
# Any other sequences there?
```

```
next(GenBankseq)
```

```
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
```

```
File "/usr/lib/python3/dist-packages/Bio/SeqIO/Interfaces.py", line 85, in __next__
    return next(self.records)
```

```
StopIteration
```

```
# What does mito contain?
```

```
type(mito)
```

```
<class 'Bio.SeqRecord.SeqRecord'>
```

```
mito
```

```
SeqRecord(seq=Seq('GTAATATTAGTATAATTTTATTATTTTAAATTTTCGATTAAAGATTTTTTATT...AGG'),
  id='NC_001861.1',
  name='NC_001861',
```


<code>description='Onchocerca volvulus strain forest mitochondrion, complete genome', dbxrefs=['BioProject:PRJNA11839'])</code>
mito.id
<code>'NC_001861.1'</code>
mito.name
<code>'NC_001861'</code>
mito.description
<code>'Onchocerca volvulus strain forest mitochondrion, complete genome'</code>

All the other information is there too... a dictionary ...

mito.annotations

```
{'molecule_type': 'DNA',  
 'topology': 'circular',  
 'data_file_division': 'INV',  
 'date': '14-SEP-2017',  
 'accessions': ['NC_001861'],  
 'sequence_version': 1,  
 'keywords': ['RefSeq'],  
 'source': 'mitochondrion Onchocerca volvulus (river blindness roundworm)',  
 'organism': 'Onchocerca volvulus',  
 'taxonomy': ['Eukaryota', 'Metazoa', 'Ecdysozoa', 'Nematoda', 'Chromadorea', 'Rhabditida', 'Spirurina', 'Spiruromorpha', 'Filarioidea', 'Onchocercidae', 'Onchocerca'],  
 'references': [Reference(title='The mitochondrial genome of Onchocerca volvulus: sequence, structure and phylogenetic analysis', ...), Reference(title='Complete sequence of mitochondrial genome of  
 'comment': 'REVIEWED REFSEQ: This record has been curated by NCBI staff. The\nreference sequence was derived from AF015193.\nCOMPLETENESS: full length.')
```

Focus of aspects of the annotation...

mito.annotations['source']

`'mitochondrion Onchocerca volvulus (river blindness roundworm)'`

mito.annotations['taxonomy']

`['Eukaryota', 'Metazoa', 'Ecdysozoa', 'Nematoda', 'Chromadorea', 'Rhabditida', 'Spirurina', 'Spiruromorpha',
'Filarioidea', 'Onchocercidae', 'Onchocerca']`

...as well as the sequence

mito_sequence = str(mito.seq)

len(mito_sequence)

`13747`

Seq(mito_sequence)

`Seq('GTAATATTAGTATAATTTTATTATTTTAAATTTTCGATTAAGATTTTATT...AGG')`

We can work on substrings of our sequence too

Top strand frame 1

Seq(mito_sequence[0:])

`Seq('GTAATATTAGTATAATTTTATTATTTTAAATTTTCGATTAAGATTTTATT...AGG')`

Top strand frame 2

Seq(mito_sequence[1:])

`Seq('TAATATTAGTATAATTTTATTATTTTAAATTTTCGATTAAGATTTTATT...AGG')`

Top strand frame 3

Seq(mito_sequence[2:])

`Seq('AATATTAGTATAATTTTATTATTTTAAATTTTCGATTAAGATTTTATTAT...AGG')`

Chose the table carefully...

Seq(mito_sequence[0:]).translate(table=1)

`Seq('VILV*FYFY*FSIKRFFIISRFFVSFLN*F*VFCSSILLFFVFFLLLFLSFINF...EEE')`

Seq(mito_sequence[0:]).translate(table=2)

`Seq('VMLV*FYFY*FSIK*FFIIS*FFVSFLNFWFVFCSSILLFFVFFLLLFLSFINF...EEE')`

Seq(mito_sequence[0:]).translate(table=3)

Seq('VMLV*FYF*FSIKRFFIISRFFVSFTNWFVFCSSILLFFVFFLLFLSFINF...EEE')

Top strand frame 2

Seq(mito_sequence[1:]).translate(table=1)

Seq('*Y*YNFIIFNRLKDFLLVGFLLVF*TDFEFFARLFCYFLFFFYCF*VLLIF...KKR')

Seq(mito_sequence[1:]).translate(table=2)

Seq('*Y*YNFIIFNRLKDFLLVGFLLVF*TDFEFFARLFCYFLFFFYCFWVLLIF...KK*')

Seq(mito_sequence[1:]).translate(table=3)

Seq('*Y*YNFIIFNRLKDFLLVGFLLVF*TDFEFFARTFCYFLFFFYCFWVLLIF...KKR')

etc.

📌 Dealing with sequence features

As we will remember (hopefully!) from a previous lecture

https://bioinfmsc8.bio.ed.ac.uk/AY25_BPSM07.html

GenBank files contain records, which contain annotation **features** ...below is an excerpt from sequence NC_001861 which includes the region coding for a ND2 gene.....

```
...
    tRNA          59..112
                  /product="tRNA-Ser"
                  /codon_recognized="AGN"
    gene          112..960
                  /gene="ND2"
                  /db_xref="GeneID:808147"
    CDS           112..960
                  /gene="ND2"
                  /codon_start=1
                  /transl_table=5
                  /product="NADH dehydrogenase subunit 2"
                  /protein_id="NP_008368.2"
                  /db_xref="GeneID:808147"
                  /translation="MLFFVFFLLFLSFINFCVVDYIVWWSIFVICTFVFVFLVGCEL
GFSSCLVNYVVIQEVCGYYFLVFDGWKLQFLLMLKSGSSPFHFWLFSVLGGLNKFV
LWFLTLQKLPYFVVLVNFCDFFFLFLFFGMIFCYFQFFLLRSYRDLLVVGSAESFNW
LLLLGIFSFNEVFVLFYYFVMFFVISYVYGGFLGFLSLEMLMFFFNVPLSITFFLK
VIVLFGSSFFVGFYYLFLFLFMPLMSLGVGYLFFLVSMMSFNCGFKYYDYFVYVLFCL
GLLSCF"
    tRNA          968..1026
                  /product="tRNA-Thr"
.... etc
```

Open the GenBank file we looked at previously

```
records = SeqIO.parse(my_gbfile, "genbank")
for seq_record in records:
    for feature in seq_record.features:
        print(feature.type)
        print(feature.location)
```

```
source
[0:13747] (+)
tRNA
[0:57] (+)
tRNA
[58:112] (+)
gene
[111:960] (+)
CDS
[111:960] (+)
tRNA
[967:1026] (+)
gene
[1025:2258] (+)
CDS
[1025:2258] (+)
gene
[2265:3912] (+)
```

[Give feedback](#)
[Ask AI for help](#)

CDS

[2265:3912] (+)

tRNA

[3920:3977] (+)

...

... LOTS MORE ...

📌 Feature qualifiers

"Features" are the things like:

```
tRNA      58..112
gene      111..960
CDS       111..960
```

"Qualifiers" are attached to features, so are things like:

```
/product="tRNA-Ser"
/codon_recognized="AGN"

or
/gene="ND2"
/db_xref="GeneID:808147"

or
/gene="ND2"
/codon_start=1
/transl_table=5
/product="NADH dehydrogenase subunit 2"
/protein_id="NP_008368.1"
/db_xref="GI:5835444"
/db_xref="GeneID:808147"
/translation="MLFFVFLLFLSFNFCVVDYIVWWSIFVICTFVFVFLVGGE
LGFSSCLVNYIYIQEVCGYFLVFDGWLQFLLMLKSGSSPFHFWLFSVLGGLNKF
VLWFLTLQKLPYFVVLVNFCDFFFLFLFFGMIFCYFQFFLLRSYRDLVVGSAESFN
WLLLLGIFSFNEVFVLFYFFVYFVMFFVISYVYGGFLGFLSLEMLMFFFNPLSITFFL
KVIVLFGSSFFVGFYFLFLFLMPLMSLGVGLFFLVSMMSFNCGFKYDYFVYVLF
IGLLSCF"
```

```
# Features have a dictionary of zero or more qualifiers, which we can read
```

```
records = SeqIO.parse(my_gbfile, "genbank")
```

```
# What is in there?
```

```
records
```

```
<Bio.SeqIO.InsdcIO.GenBankIterator object at 0x7f5b37caad00>
```

```
type(records)
```

```
<class 'Bio.SeqIO.InsdcIO.GenBankIterator'>
```

```
len(records)
```

```
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
```

```
TypeError: object of type 'GenBankIterator' has no len()
```

```
# Can we see it as a list?
```

```
list(records)
```

```
[
  SeqRecord(seq=Seq('GTAATATTAGTATAATTTATTATTTTAAATTTTCGATTAAAGATTTTTTATT...AGG'),
    id='NC_001861.1',
    name='NC_001861',
    description='Onchocerca volvulus strain forest mitochondrion, complete genome',
    dbxrefs=['BioProject:PRJNA11839'])
]
```

```
# Can we see it as a list again?
```

```
list(records)
```

```
[]
```

```
records = SeqIO.parse(my_gbfile, "genbank")
```

```
len(list(records))  
1  
len(list(records))  
0  
list(SeqIO.parse(my_gbfile, "genbank"))  
[  
  SeqRecord(seq=Seq('GTAATATTAGTATAATTTTATTATTTTAAATTTTCGATTAAGATTTTTATT...AGG'),  
    id='NC_001861.1',  
    name='NC_001861',  
    description='Onchocerca volvulus strain forest mitochondrion, complete genome',  
    dbxrefs=['BioProject:PRJNA11839'])  
]
```

```
len(list(SeqIO.parse(my_gbfile, "genbank")))  
1
```

```
# Access the next (first) record  
records = SeqIO.parse(my_gbfile, "genbank")  
MyGenbank_record = next(records)
```

```
# What do we have: look with DOT TAB TAB  
MyGenbank_record.  


|                              |                           |                |
|------------------------------|---------------------------|----------------|
| MyGenbank_record.annotations | MyGenbank_record.features | MyGenbank_reco |
| MyGenbank_record.dbxrefs     | MyGenbank_record.format(  | MyGenbank_reco |
| MyGenbank_record.description | MyGenbank_record.id       | MyGenbank_reco |


```

```
# What do we have: look with DOT TAB TAB (different this time!)  
len(MyGenbank_record.  


|                              |                           |                |
|------------------------------|---------------------------|----------------|
| MyGenbank_record.annotations | MyGenbank_record.features | MyGenbank_reco |
| MyGenbank_record.dbxrefs     | MyGenbank_record.format(  | MyGenbank_reco |
| MyGenbank_record.description | MyGenbank_record.id       | MyGenbank_reco |


```

```
len(MyGenbank_record.description)  
64  
MyGenbank_record.description  
'Onchocerca volvulus strain forest mitochondrion, complete genome'  
len(MyGenbank_record.dbxrefs)  
1  
len(MyGenbank_record.seq)  
13747  
len(MyGenbank_record.features)  
50
```

```
# Let's loop through each of the features, looking for the gene "ND2"  
wanted="ND2"  
counter=0  
for each_feat in MyGenbank_record.features :  
    counter += 1  
    if each_feat.qualifiers.get('gene') :  
        genename_string="".join(str(e) for e in each_feat.qualifiers.get("gene"))  
        if genename_string == wanted :  
            print("\n### ",counter,wanted,"\n",each_feat,each_feat.qualifiers['gene'])
```

```
### 4 ND2
  type: gene
location: [111:960] (+)
qualifiers:
  Key: db_xref, Value: ['GeneID:808147']
  Key: gene, Value: ['ND2']
  ['ND2']

### 5 ND2
  type: CDS
location: [111:960] (+)
qualifiers:
  Key: codon_start, Value: ['1']
  Key: db_xref, Value: ['GeneID:808147']
  Key: gene, Value: ['ND2']
  Key: product, Value: ['NADH dehydrogenase subunit 2']
  Key: protein_id, Value: ['NP_008368.2']
  Key: transl_table, Value: ['5']
  Key: translation, Value: ['MLFFVFFLLFLSFINFCVVDYIVWWSIFVICTFVFVFLVGGELGFSSCLVNYVVIQEV
  ['ND2']
```

```
# Let's have a look at the first protein CoDing Segment (CDS),
# gene ND2, which we now know was the 5th feature (Python index position 4)
```

```
my_ND2_feature = MyGenbank_record.features[4]
my_ND2_feature
SeqFeature(SimpleLocation(ExactPosition(111), ExactPosition(960), strand=1), type='CDS', qualifiers=...)
my_ND2_feature.type
'CDS'
```

```
# What are the qualifiers?
```

```
my_ND2_feature.qualifiers
{'gene': ['ND2'],
 'codon_start': ['1'],
 'transl_table': ['5'],
 'product': ['NADH dehydrogenase subunit 2'],
 'protein_id': ['NP_008368.2'],
 'db_xref': ['GeneID:808147'],
 'translation': ['MLFFVFFLLFLSFINFCVVDYIVWWSIFVICTFVFVFLVGGELGFSSCLVNYVVIQEVCGYYFLVFDGWK']
```

```
type(my_ND2_feature.qualifiers)
```

```
<class 'dict'>
```

```
# We could do it this way if we just want to know the keys...
```

```
print("\n".join(list(my_ND2_feature.qualifiers)))
```

```
gene
codon_start
transl_table
product
protein_id
db_xref
translation
```

```
What do we have: look with DOT TAB TAB
```

```
my_ND2_feature.
```

my_ND2_feature.extract(	my_ND2_feature.location_operator	my_ND2_feature.ref_db
my_ND2_feature.id	my_ND2_feature.qualifiers	my_ND2_feature.strand
my_ND2_feature.location	my_ND2_feature.ref	my_ND2_feature.transla

📌 Getting the sequence for a feature

Extracting a feature creates a new record object

```
new_ND2_record = my_ND2_feature.extract(MyGenbank_record)
```

new_ND2_record

```
SeqRecord(  
    seq=Seq('TTGTTATTTTTGTTTTTTTTTATTATTGTTTTGAGTTTATTAATTTTGT...TAG'),  
    id='NC_001861.1',  
    name='NC_001861',  
    description='Onchocerca volvulus strain forest mitochondrion, complete genome',  
    dbxrefs=[])
```

What do we have now: look with DOT TAB TAB again

new_ND2_record.

new_ND2_record.annotations	new_ND2_record.format(	new_ND2_record.name
new_ND2_record.dbxrefs	new_ND2_record.id	new_ND2_record.rev
new_ND2_record.description	new_ND2_record.letter_annotations	new_ND2_record.seq
new_ND2_record.features	new_ND2_record.lower(	new_ND2_record.tra

Get the actual sequence of a feature

new_ND2_record.seq

```
Seq('TTGTTATTTTTGTTTTTTTTTATTATTGTTTTGAGTTTATTAATTTTGT...TAG')
```

len(new_ND2_record.seq)

849

print(new_ND2_record.seq)

```
ATTGTTGATTTTTTTGTTTTTTTATTATTGTTTTGAGTTTATTAATTTTGTGTTGATTATATTGTTGGTGAGGATTTTGTATTGTTGACTTTTGTGTTTTCTGTTGGTGAGTTGAGTTAGAAAGTTGTTGGTTAAATTATTATGTTATTCAAGAAAGTTTGGTTATTATTTTGGTTTTGATGGTTGGAAGTTGCAATTTT
```

Translate it, and show 30 amino acids per line!

import re

```
print("\n".join(re.findall(r'.{1,30}', str(new_ND2_record.seq.translate(table=5)))))
```

```
LLFFVFFLLLLFSLFINFCVVDYIVWWSIFV  
ICTFVFVFLVGGELGFSSCLVNYVYIQEVC  
GYYFLVFDGWKLQFLLMLKSGSSPFHFWL  
FSVLGGLNKFVFLWFLTLQKLPYFVVLVNF  
CGDFFFLFLFFGMIFCYFQFFLLRSYRDL  
VVGSAESFNWLLLLGIFSFNEVFVLFYFY  
FVMFFVISYVYGGFLGFLSLEMLMFFFNVP  
LSITFFFLKVIVLFGSFFVGFYFLFLLEFM  
PLMSLGVGYLFFLVSMMSFNCGFKYYDYFV  
YVLFICIGLLSCF*
```

That long command.... formatted for ease of viewing only

```
print(  
    "\n".join(  
        re.findall(r'.{1,30}',  
        str(  
            new_ND2_record.seq.translate(table=5)  
        )  
    )  
)
```

```
# fifth: print the joined things to screen  
# fourth: join the findall list elements with  
# third: find 1-30 character stretches in the  
# second: make the translation into a string  
# first: get and translate the sequence using  
# close str  
# close findall  
# close join  
# close print
```

Use a different translation table

```
print("\n".join(re.findall(r'.{1,30}', str(new_ND2_record.seq.translate(table=1)))))
```

```
LLFFVFFLLLLFLSFINFCVVDYIVWWRIFV
ICTFVFVFLVGGELGFRSCLVNYYVIEVC
GYYFLVFDGWKLQFLLLMLKSGSSPFHF * L
FSVLGGLNKWFVLWFLTLQKLPYFVVLVNF
CGDFFFLFLFFGMIFCYFQFFLLRSYRDLL
VVGSAESFNWLLLLGIFSFNEVFVLFFFYY
FVIEFFVISYVYGGFLGFLSLEILMFFFNVP
LRITFFLKVIVLFGSSFFVGFYYLFLLLFM
PLMSLGVGYLFFLVSMMSFNCGFKYYDYFV
YVLCIGLLSCF*
```

Use a different translation table, translate to first stop

```
print("\n".join(re.findall(r'.{1,30}', str(new_ND2_record.seq.translate(table=1,to_stop=TRUE)))))
```

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

NameError: name 'TRUE' is not defined. Did you mean: 'True'?

Correct syntax this time...!

```
print("\n".join(re.findall(r'.{1,30}', str(new_ND2_record.seq.translate(table=1,to_stop=True)))))
```

```
LLFFVFFLLLLFLSFINFCVVDYIVWWRIFV
ICTFVFVFLVGGELGFRSCLVNYYVIEVC
GYYFLVFDGWKLQFLLLMLKSGSSPFHF
```

🔍 Searching the NCBI databases using BioPython's Entrez utilities

Note that these are similar but not identical to NCBI's EDirect (e.g. esearch, efetch, efilter, xtract etc.) that you have installed ([EDirect help page](#)).

```
# Import the Entrez module
```

```
from Bio import Entrez
```

```
# What do we have: look with DOT TAB TAB
```

```
Entrez.
```

Entrez.HTTPError(	Entrez.email	Entrez.read(
Entrez.Request(	Entrez.epost(	Entrez.sleep_between_tries
Entrez.URLError(	Entrez.esearch(	Entrez.time
Entrez.api_key	Entrez.espell(	Entrez.tool
Entrez.ecitmatch(	Entrez.esummary(	Entrez.urlencode(
Entrez.efetch(	Entrez.function_with_previous(	Entrez.urlopen(
Entrez.egquery(	Entrez.io	Entrez.warnings
Entrez.einfo(	Entrez.max_tries	
Entrez.elink(	Entrez.parse(	

```
# First we must set our email address (NCBI requirement)
```

```
Entrez.email = "s123456@ed.ac.uk"
```

```
# As you may have found out recently (doing the optional ICA...)
```

... having an NCBI account and an API key helps access sometimes...

When lots of people are all doing their `esearch` commands, NCBI sees them as all coming from the IP address of the `msc8` or `msc7` server, then assuming it is one individual.

When it gets to be more than 3 requests per second, it restricts temporarily, so if you are unlucky to be doing a search then, your code will fail (and it won't be your fault!).

There is a solution: `get an API key` which entitles you as an individual user (on any server) up to 10 requests per second.

Where can you get the key? Check [this NCBI web page](#) and [the Entrez Direct help manual](#) for how to get one. You will need to [get an NCBI account](#).

Once you get it, then make life easier for yourself by adding it as a variable to your login files:

```
echo "export NCBI_API_KEY=f8230...whatever...6364c07" >> ~/.bashrc
```

```
echo "export NCBI_API_KEY=f8230...whatever...6364c07" >> ~/.bash_profile
```

and then logout and log back in again, then, to confirm all is OK, check that you get this response:

```
echo ${NCBI_API_KEY}
```

```
f8230...whatever...6364c07
```

If you get the above, then the NCBI issue should be less frequent (I hope!!)

See <https://www.ncbi.nlm.nih.gov/books/NBK179288/> for more details on this (and other things....!)

```
# Need to access the value of our NCBI_API_KEY and use it here ...  
import subprocess  
api_key=subprocess.check_output("echo ${NCBI_API_KEY}", shell=True).rstrip()  
api_key  
b'f8230825b.....86364c07'
```

```
# That b' looks wierd...  
# bytes was a new data type introduced in Python 3  
type(api_key)  
< class 'bytes'>
```

Remember, we could convert `bytes` to `str` by using `decode` with the encoding method.
The default character encoding is utf-8.
What is utf? It is the Unicode stuff we did way back in [Lecture 1](#)!
UTF stands for [Unicode Transformation Format](#).

```
api_key  
b'f8230825b.....86364c07'  
api_key.decode('utf-8')  
'f8230825b.....86364c07'
```

```
# OK, so we can carry on now...  
Entrez.api_key =api_key.decode('utf-8')
```

```
# Now we can do things, such as get info about all the databases  
Entrez.einfo()  
<http.client.HTTPResponse object at 0x7f5b37b8d430>  
IO_link = Entrez.einfo()  
record = Entrez.read(IO_link)
```

```
# record is now a dictionary with ONLY ONE key  
record.keys()  
dict_keys(['DbList'])
```

```
# The values stored in this key is the list of database names.  
record.values()  
dict_values(['pubmed', 'protein', 'nucore', 'ipg', 'nucleotide', 'structure', 'genome', 'annotinfo', 'assembly',  
'bioproject', 'biosample', 'blastdbinfo', 'books', 'cdd', 'clinvar', 'gap', 'gapplus', 'grasp', 'dbvar', 'gene', 'gds',  
'geoprofiles', 'homologene', 'medgen', 'mesh', 'nlmcatalog', 'omim', 'orgtrack', 'pmc', 'popset',  
'proteinclusters', 'pcassay', 'protfam', 'pccompound', 'pcsubstance', 'seqannot', 'snp', 'sra', 'taxonomy',  
'biocollections', 'gtr'])
```

```
# For each database, we can use einfo() again to obtain more information  
Entrez.einfo(db="pubmed")  
<http.client.HTTPResponse object at 0x7f5d5620cbb0>
```

```
IO_link = Entrez.einfo(db="pubmed")  
record = Entrez.read(IO_link)
```

type(record)

<class 'Bio.Entrez.Parser.DictionaryElement' >

record.keys()

dict_keys(['DbInfo'])

record['DbInfo'].keys()

dict_keys(['DbName', 'MenuName', 'Description', 'DbBuild', 'Count', 'LastUpdate', 'FieldList', 'LinkList'])

record.values()

dict_values([{'DbName': 'pubmed', 'MenuName': 'PubMed', 'Description': 'PubMed bibliographic record', 'DbBuild': 'Build-2025.11.13.01.10', 'Count': '39657568', 'LastUpdate': '2025/11/13 01:10', 'FieldList': [{'Name': 'ALL', 'FullName': 'All Fields', 'Description': 'All terms from all searchable fields', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'N', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'UID', 'FullName': 'UID', 'Description': 'Unique number assigned to publication', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'Y', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'Y'}, {'Name': 'FILT', 'FullName': 'Filter', 'Description': 'Limits the records', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'TITL', 'FullName': 'Title', 'Description': 'Words in title of publication', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'N', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'MESH', 'FullName': 'MeSH Terms', 'Description': 'Medical Subject Headings assigned to publication', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'Y', 'IsHidden': 'N'}, {'Name': 'MAJR', 'FullName': 'MeSH Major Topic', 'Description': 'MeSH terms of major importance to publication', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'Y', 'IsHidden': 'N'}, {'Name': 'JOUR', 'FullName': 'Journal', 'Description': 'Journal abbreviation of publication', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'AFFL', 'FullName': 'Affiliation', 'Description': 'Author's institutional affiliation and address', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'N', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'ECNO', 'FullName': 'EC/RN Number', 'Description': 'EC number for enzyme or CAS registry number', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'SUBS', 'FullName': 'Supplementary Concept', 'Description': 'CAS chemical name or MEDLINE Substance Name', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'PDAT', 'FullName': 'Date - Publication', 'Description': 'Date of publication', 'TermCount': '', 'IsDate': 'Y', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'EDAT', 'FullName': 'Date - Entry', 'Description': 'Date publication first accessible through Entrez', 'TermCount': '', 'IsDate': 'Y', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'VOL', 'FullName': 'Volume', 'Description': 'Volume number of publication', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'PAGE', 'FullName': 'Pagination', 'Description': 'Page number(s) of publication', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'PTYP', 'FullName': 'Publication Type', 'Description': 'Type of publication (e.g., review)', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'Y', 'IsHidden': 'N'}, {'Name': 'LANG', 'FullName': 'Language', 'Description': 'Language of publication', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'ISS', 'FullName': 'Issue', 'Description': 'Issue number of publication', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'SUBH', 'FullName': 'MeSH Subheading', 'Description': 'Additional specificity for MeSH term', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'Y', 'IsHidden': 'N'}, {'Name': 'SI', 'FullName': 'Secondary Source ID', 'Description': 'Cross-reference from publication to other databases', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'MHDA', 'FullName': 'Date - MeSH', 'Description': 'Date publication was indexed with MeSH terms', 'TermCount': '', 'IsDate': 'Y', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'TIAB', 'FullName': 'Title/Abstract', 'Description': 'Free text associated with Abstract/Title', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'N', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'OTRM', 'FullName': 'Other Term', 'Description': 'Other terms associated with publication', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'N', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'COLN', 'FullName': 'Author - Corporate', 'Description': 'Corporate Author of publication', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'N', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'CNTY', 'FullName': 'Place of Publication', 'Description': 'Country of publication', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'N', 'Hierarchy': 'N', 'IsHidden': 'Y'}, {'Name': 'PAPX', 'FullName': 'Pharmacological Action', 'Description': 'MeSH pharmacological action pre-explosions', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'GRNT', 'FullName': 'Grants and Funding', 'Description': 'NIH Grant Numbers', 'TermCount': ''},

'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'MDAT', 'FullName': 'Date - Modification', 'Description': 'Date of last modification', 'TermCount': '', 'IsDate': 'Y', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'CDAT', 'FullName': 'Date - Completion', 'Description': 'Date of completion', 'TermCount': '', 'IsDate': 'Y', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'PID', 'FullName': 'Publisher ID', 'Description': 'Publisher ID', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'Y'}, {'Name': 'FAUT', 'FullName': 'Author - First', 'Description': 'First Author of publication', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'FULL', 'FullName': 'Author', 'Description': 'Full Author Name(s) of publication', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'FINV', 'FullName': 'Investigator', 'Description': 'Full name of investigator', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'TT', 'FullName': 'Transliterated Title', 'Description': 'Words in transliterated title of publication', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'N', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'LAUT', 'FullName': 'Author - Last', 'Description': 'Last Author of publication', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'PPDT', 'FullName': 'Print Publication Date', 'Description': 'Date of print publication', 'TermCount': '', 'IsDate': 'Y', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'Y'}, {'Name': 'EPDT', 'FullName': 'Electronic Publication Date', 'Description': 'Date of Electronic publication', 'TermCount': '', 'IsDate': 'Y', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'Y'}, {'Name': 'LID', 'FullName': 'Location ID', 'Description': 'ELocation ID', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'CRDT', 'FullName': 'Date - Create', 'Description': 'Date publication first accessible through Entrez', 'TermCount': '', 'IsDate': 'Y', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'BOOK', 'FullName': 'Book', 'Description': 'ID of the book that contains the document', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'ED', 'FullName': 'Editor', 'Description': 'Section's Editor', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'ISBN', 'FullName': 'ISBN', 'Description': 'ISBN', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'PUBN', 'FullName': 'Publisher', 'Description': 'Publisher's name', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'AUCL', 'FullName': 'Author Cluster ID', 'Description': 'Author Cluster ID', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'Y'}, {'Name': 'EID', 'FullName': 'Extended PMID', 'Description': 'Extended PMID', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'Y'}, {'Name': 'DSO', 'FullName': 'DSO', 'Description': 'Additional text from the summary', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'N', 'Hierarchy': 'N', 'IsHidden': 'Y'}, {'Name': 'AUID', 'FullName': 'Author - Identifier', 'Description': 'Author Identifier', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'PS', 'FullName': 'Subject - Personal Name', 'Description': 'Personal Name as Subject', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'COIS', 'FullName': 'Conflict of Interest Statements', 'Description': 'Conflict of Interest Statements', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'N', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'WORD', 'FullName': 'Text Word', 'Description': 'Free text associated with publication', 'TermCount': '', 'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'N', 'Hierarchy': 'N', 'IsHidden': 'N'}, {'Name': 'P1DAT', 'FullName': 'P1DAT', 'Description': 'Date publication first accessible through Solr', 'TermCount': '', 'IsDate': 'Y', 'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'Y'}], 'LinkList': [{'Name': 'pubmed_assembly', 'Menu': 'Assembly', 'Description': 'Assembly', 'DbTo': 'assembly'}, {'Name': 'pubmed_bioproject', 'Menu': 'Project Links', 'Description': 'Related Projects', 'DbTo': 'bioproject'}, {'Name': 'pubmed_biosample', 'Menu': 'BioSample Links', 'Description': 'BioSample links', 'DbTo': 'biosample'}, {'Name': 'pubmed_biosystems', 'Menu': 'BioSystem Links', 'Description': 'BioSystems', 'DbTo': 'biosystems'}, {'Name': 'pubmed_books_refs', 'Menu': 'Cited in Books', 'Description': 'PubMed links associated with Books', 'DbTo': 'books'}, {'Name': 'pubmed_cdd', 'Menu': 'Conserved Domain Links', 'Description': 'Link to related CDD entry', 'DbTo': 'cdd'}, {'Name': 'pubmed_clinvar', 'Menu': 'ClinVar', 'Description': 'Clinical variations associated with publication', 'DbTo': 'clinvar'}, {'Name': 'pubmed_clinvar_calculated', 'Menu': 'ClinVar (calculated)', 'Description': 'Clinical variations calculated to be associated with publication', 'DbTo': 'clinvar'}, {'Name': 'pubmed_dbvar', 'Menu': 'dbVar', 'Description': 'Link from PubMed to dbVar', 'DbTo': 'dbvar'}, {'Name': 'pubmed_gap', 'Menu': 'dbGaP Links', 'Description': 'Related dbGaP record', 'DbTo': 'gap'}, {'Name': 'pubmed_gds', 'Menu': 'GEO DataSet Links', 'Description': 'Related GEO DataSets', 'DbTo': 'gds'}, {'Name': 'pubmed_gene', 'Menu': 'Gene Links', 'Description': 'Link to related Genes', 'DbTo': 'gene'}, {'Name': 'pubmed_gene_bookrecords', 'Menu': 'Gene (from Bookshelf)', 'Description': 'Gene records in this citation', 'DbTo': 'gene'}, {'Name': 'pubmed_gene_citedinomim', 'Menu': 'Gene (OMIM) Links', 'Description': 'PubMed links to Gene derived from pubmed_omim_cited links', 'DbTo':

'gene'}, {'Name': 'pubmed_gene_pmc_nucleotide', 'Menu': 'Gene (nucleotide/PMC)', 'Description': 'Records in Gene identified from shared sequence and PMC links.', 'DbTo': 'gene'}, {'Name': 'pubmed_gene_rif', 'Menu': 'Gene (GeneRIF) Links', 'Description': 'Link to Gene for the GeneRIF subcategory', 'DbTo': 'gene'}, {'Name': 'pubmed_genome', 'Menu': 'Genome Links', 'Description': 'Published genome sequences', 'DbTo': 'genome'}, {'Name': 'pubmed_geoprofiles', 'Menu': 'GEO Profile Links', 'Description': 'GEO records associated with pubmed record', 'DbTo': 'geoprofiles'}, {'Name': 'pubmed_homologene', 'Menu': 'HomoloGene Links', 'Description': 'Related HomoloGene', 'DbTo': 'homologene'}, {'Name': 'pubmed_medgen', 'Menu': 'MedGen', 'Description': 'Related information in MedGen', 'DbTo': 'medgen'}, {'Name': 'pubmed_medgen_bookshelf_cited', 'Menu': 'MedGen (Bookshelf cited)', 'Description': 'Related records in MedGen based on citations in GeneReviews and Medical Genetics Summaries', 'DbTo': 'medgen'}, {'Name': 'pubmed_medgen_genereviews', 'Menu': 'MedGen (GeneReviews)', 'Description': 'Related MedGen records', 'DbTo': 'medgen'}, {'Name': 'pubmed_medgen_omim', 'Menu': 'MedGen (OMIM)', 'Description': 'Related information in MedGen (OMIM)', 'DbTo': 'medgen'}, {'Name': 'pubmed_nuccore', 'Menu': 'Nucleotide Links', 'Description': 'Published Nucleotide sequences', 'DbTo': 'nuccore'}, {'Name': 'pubmed_nuccore_refseq', 'Menu': 'Nucleotide (RefSeq) Links', 'Description': 'Link to Nucleotide RefSeqs', 'DbTo': 'nuccore'}, {'Name': 'pubmed_nuccore_weighted', 'Menu': 'Nucleotide (Weighted) Links', 'Description': 'Links to nuccore', 'DbTo': 'nuccore'}, {'Name': 'pubmed_omim_bookrecords', 'Menu': 'OMIM (from Bookshelf)', 'Description': 'OMIM records in this citation', 'DbTo': 'omim'}, {'Name': 'pubmed_omim_calculated', 'Menu': 'OMIM (calculated) Links', 'Description': 'OMIM (calculated) Links', 'DbTo': 'omim'}, {'Name': 'pubmed_omim_cited', 'Menu': 'OMIM (cited) Links', 'Description': 'OMIM (cited) Links', 'DbTo': 'omim'}, {'Name': 'pubmed_pccassay', 'Menu': 'PubChem BioAssay', 'Description': 'Related PubChem BioAssay', 'DbTo': 'pcassay'}, {'Name': 'pubmed_pccompound', 'Menu': 'PubChem Compound', 'Description': 'Related PubChem Compound', 'DbTo': 'pccompound'}, {'Name': 'pubmed_pccompound_mesh', 'Menu': 'PubChem Compound (MeSH Keyword)', 'Description': 'Related PubChem Compound via MeSH', 'DbTo': 'pccompound'}, {'Name': 'pubmed_pccompound_publisher', 'Menu': 'PubChem Compound (Publisher)', 'Description': 'Publisher deposited structures linked to PubChem Compound', 'DbTo': 'pccompound'}, {'Name': 'pubmed_pcsubstance', 'Menu': 'PubChem Substance Links', 'Description': 'Related PubChem Substance', 'DbTo': 'pcsubstance'}, {'Name': 'pubmed_pcsubstance_bookrecords', 'Menu': 'PubChem Substance (from Bookshelf)', 'Description': 'Structures in the PubChem Substance database in this citation', 'DbTo': 'pcsubstance'}, {'Name': 'pubmed_pcsubstance_publisher', 'Menu': 'PubChem Substance (Publisher)', 'Description': 'PubChem Substances supplied by publisher', 'DbTo': 'pcsubstance'}, {'Name': 'pubmed_pmc', 'Menu': 'PMC Links', 'Description': 'Free full text articles in PMC', 'DbTo': 'pmc'}, {'Name': 'pubmed_pmc_bookrecords', 'Menu': 'References in PMC for this Bookshelf citation', 'Description': 'Full text of articles in PubMed Central cited in this record', 'DbTo': 'pmc'}, {'Name': 'pubmed_pmc_embargo', 'Menu': '', 'Description': 'Embargoed PMC article associated with PubMed', 'DbTo': 'pmc'}, {'Name': 'pubmed_pmc_local', 'Menu': '', 'Description': 'Free full text articles in PMC', 'DbTo': 'pmc'}, {'Name': 'pubmed_pmc_refs', 'Menu': 'Cited in PMC', 'Description': 'PubMed links associated with PMC', 'DbTo': 'pmc'}, {'Name': 'pubmed_popset', 'Menu': 'PopSet Links', 'Description': 'Published population set', 'DbTo': 'popset'}, {'Name': 'pubmed_probe', 'Menu': 'Probe Links', 'Description': 'Related Probe entry', 'DbTo': 'probe'}, {'Name': 'pubmed_protein', 'Menu': 'Protein Links', 'Description': 'Published protein sequences', 'DbTo': 'protein'}, {'Name': 'pubmed_protein_refseq', 'Menu': 'Protein (RefSeq) Links', 'Description': 'Link to Protein RefSeqs', 'DbTo': 'protein'}, {'Name': 'pubmed_protein_weighted', 'Menu': 'Protein (Weighted) Links', 'Description': 'Links to protein', 'DbTo': 'protein'}, {'Name': 'pubmed_proteinclusters', 'Menu': 'Protein Cluster Links', 'Description': 'Related Protein Clusters', 'DbTo': 'proteinclusters'}, {'Name': 'pubmed_protfam', 'Menu': 'Protein Family Models', 'Description': 'Protein family models supported by a particular publication', 'DbTo': 'protfam'}, {'Name': 'pubmed_pubmed', 'Menu': 'Similar articles', 'Description': 'Similar PubMed articles, obtained by matching text and MeSH terms', 'DbTo': 'pubmed'}, {'Name': 'pubmed_pubmed_alsoviewed', 'Menu': 'Articles frequently viewed together', 'Description': 'Articles frequently viewed together', 'DbTo': 'pubmed'}, {'Name': 'pubmed_pubmed_bookrecords', 'Menu': 'References for this Bookshelf citation', 'Description': 'PubMed abstracts for articles cited in this record', 'DbTo': 'pubmed'}, {'Name': 'pubmed_pubmed_refs', 'Menu': 'References for PMC Articles', 'Description': 'References for this PMC Article', 'DbTo': 'pubmed'}, {'Name': 'pubmed_snp', 'Menu': 'SNP Links', 'Description': 'PubMed to SNP links', 'DbTo': 'snp'}, {'Name': 'pubmed_snp_cited', 'Menu': 'SNP (Cited)', 'Description': 'Related SNP (Cited) records', 'DbTo': 'snp'}, {'Name': 'pubmed_sra', 'Menu': 'SRA Links', 'Description': 'Links to Short Read Archive Experiments', 'DbTo': 'sra'}, {'Name': 'pubmed_structure', 'Menu': 'Structure Links', 'Description': 'Published 3D structures', 'DbTo': 'structure'}, {'Name': 'pubmed_taxonomy_entrez', 'Menu': 'Taxonomy via GenBank', 'Description': 'Related Taxonomy entry computed using other Entrez links', 'DbTo': 'taxonomy'}}]]))


```
record["DbInfo"]["Description"]
```

```
'PubMed bibliographic record'
```

```
record["DbInfo"]["Count"]
```

```
'39657568'
```

```
record["DbInfo"]["LastUpdate"]
```

```
'2025/11/13 01:10'
```

```
len(record['DbInfo']['FieldList'])
```

```
50
```

```
record['DbInfo']['FieldList'][0]
```

```
{'Name': 'ALL', 'FullName': 'All Fields', 'Description': 'All terms from all searchable fields', 'TermCount': '',  
'IsDate': 'N', 'IsNumerical': 'N', 'SingleToken': 'N', 'Hierarchy': 'N', 'IsHidden': 'N'}
```

```
record['DbInfo']['FieldList'][1]
```

```
{'Name': 'UID', 'FullName': 'UID', 'Description': 'Unique number assigned to publication', 'TermCount': '',  
'IsDate': 'N', 'IsNumerical': 'Y', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'Y'}
```

```
record['DbInfo']['FieldList'][2]
```

```
{'Name': 'FILT', 'FullName': 'Filter', 'Description': 'Limits the records', 'TermCount': '', 'IsDate': 'N',  
'IsNumerical': 'N', 'SingleToken': 'Y', 'Hierarchy': 'N', 'IsHidden': 'N'}
```

...etc...

A quick example...

```
Entrez.read(Entrez.esearch(db="pubmed",term="Ivens A[author]"))
```

```
{'Count': '197', 'RetMax': '20', 'RetStart': '0', 'IdList': ['41005611', '40527318', '40214640', '40138719', '39149912', '38408115', '37616070']}
```

```
# We can also use the Entrez object to search GenBank for sequence records.
```

```
# We need three arguments:
```

```
# name of the database ,
```

```
# term we are searching for, and
```

```
# retmax "return maximum" of results
```

```
# First we define our search
```

```
mysearch = Entrez.esearch(db="nucleotide", term="COX1", retmax="1000")
```

```
mysearch
```

```
<http.client.HTTPResponse object at 0x7f5d561d3ee0>
```

```
# Then we actually do the search (like using efetch )
```

```
result = Entrez.read(mysearch)
```

```
type(result)
```

```
<class 'Bio.Entrez.Parser.DictionaryElement'>
```

```
# What do we have in the dictionary?
```

```
result.keys()
```

```
dict_keys(['Count', 'RetMax', 'RetStart', 'IdList', 'TranslationSet', 'TranslationStack', 'QueryTranslation'])
```

```
# How many records did we find in total?
```

```
print(result['Count'])
```

```
1078946
```



```
# What are the GI/accession numbers of the first 5?
```

```
print(result['IdList'][0:5])
```

```
['3101929610', '3101906365', '3101906363', '3101874138', '3101874136']
```

We can also use the Entrez object to download a single record (if we know the ID) and turn it into an object...

```
# Let's get that ND2 gene!
```

```
genbank = Entrez.efetch(db="nucleotide", id='NP_008368.2', rettype="gb", retmode="text")
```

```
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
```

```
File "/usr/lib/python3/dist-packages/Bio/Entrez/__init__.py", line 197, in efetch
```

```
    return _open(request)
```

```
    ^^^^^^^^^^^^^^^^^
```

```
File "/usr/lib/python3/dist-packages/Bio/Entrez/__init__.py", line 623, in _open
```

```
    handle = urlopen(request)
```

```
    ^^^^^^^^^^^^^^^^^
```

```
File "/usr/lib/python3.12/urllib/request.py", line 215, in urlopen
```

```
    return opener.open(url, data, timeout)
```

```
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/usr/lib/python3.12/urllib/request.py", line 521, in open
```

```
    response = meth(req, response)
```

```
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/usr/lib/python3.12/urllib/request.py", line 630, in http_response
```

```
    response = self.parent.error(
```

```
    ^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/usr/lib/python3.12/urllib/request.py", line 559, in error
```

```
    return self._call_chain(*args)
```

```
    ^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/usr/lib/python3.12/urllib/request.py", line 492, in _call_chain
```

```
    result = func(*args)
```

```
    ^^^^^^^^^^^^^
```

```
File "/usr/lib/python3.12/urllib/request.py", line 639, in http_error_default
```

```
    raise HTTPError(req.full_url, code, msg, hdrs, fp)
```

```
urllib.error.HTTPError: HTTP Error 400: Bad Request
```

url = "Uniform Resource Locator"

```
# Oops, try again using the correct database!
```

```
genbank = Entrez.efetch(db="protein", id='NP_008368.2', rettype="gb", retmode="text")
```

```
record = genbank.read()
```

```
print(record)
```

```
LOCUS      ND2 13186                282 aa                linear    INV 03-APR-2023
DEFINITION NADH dehydrogenase subunit 2 (mitochondrion) [Onchocerca volvulus].
ACCESSION  NP_008368
VERSION    NP_008368.2
DBLINK     BioProject: PRJNA927338
DBSOURCE   REFSEQ: accession NC_001861.1
KEYWORDS   RefSeq.
SOURCE     mitochondrion Onchocerca volvulus (river blindness roundworm)
  ORGANISM Onchocerca volvulus
            Eukaryota; Metazoa; Ecdysozoa; Nematoda; Chromadorea; Rhabditida;
            Spirurina; Spiruromorpha; Filarioidea; Onchocercidae; Onchocerca.
REFERENCE  1 (residues 1 to 282)
  AUTHORS  Keddie,E.M., Higazi,T. and Unnasch,T.R.
  TITLE    The mitochondrial genome of Onchocerca volvulus: sequence,
            structure and phylogenetic analysis
  JOURNAL  Mol. Biochem. Parasitol. 95 (1), 111-127 (1998)
  PUBMED   9763293
REFERENCE  2 (residues 1 to 282)
  AUTHORS  Keddie,E.M. and Unnasch,T.R.
  TITLE    Complete sequence of mitochondrial genome of Onchocerca volvulus
  JOURNAL  Unpublished
REFERENCE  3 (residues 1 to 282)
  CONSRM   NCBI Genome Project
  TITLE    Direct Submission
  JOURNAL  Submitted (08-SEP-1999) National Center for Biotechnology
            Information, NIH, Bethesda, MD 20894, USA
REFERENCE  4 (residues 1 to 282)
  AUTHORS  Keddie,E.M. and Unnasch,T.R.
```

TITLE Direct Submission
JOURNAL Submitted (18-JUL-1997) Geographic Medicine, University of Alabama at Birmingham, BBRB Box 7, Birmingham, AL 35294-2170, USA
COMMENT PROVISIONAL REFSEQ: This record has not yet been subject to final NCBI review. The reference sequence is identical to AAC61610.

On Sep 14, 2017 this sequence version replaced NP_008368.1.
Method: conceptual translation.
FEATURES
 Location/Qualifiers
 source
 1..282
 /organism="Onchocerca volvulus"
 /organelle="mitochondrion"
 /strain="forest"
 /db_xref="taxon:6282"
 /common="river blindness roundworm"
 Protein
 1..282
 /product="NADH dehydrogenase subunit 2"
 /calculated_mol_wt=33499
 Region
 13..282
 /region_name="ND2"
 /note="NADH dehydrogenase subunit 2; Provisional;
 MTH00091"
 /db_xref="CDD:177154"
 CDS
 1..282
 /gene="ND2"
 /coded_by="NC_001861.1:112..960"
 /transl_table=5
 /db_xref="GeneID:808147"
ORIGIN
 1 mlfvfffl1l flsfincfv dyivwvsifv ictfvfvflv ggelfgsscl vnyyviqevc
 61 gyyflvfdgw klqflllmlk sgsspfhfwl fsvlgglnkw fvlwfltlqk lpyfvvlvnf
 121 cgdffflflf fgmifcyfqf flrsyrdll vvgsaesfn llllgifsfn evfvlffyy
 181 fvmffvisyv yggflgflsl emlmfffnvp lsitfflkvi vlfgssffvg fyylflllfm
 241 plmslgvgyl fflvsmsfn cgfkyydyfv yvlfciglls cf
//

🔍 Searching the GenBank database: multiple sequences

Let's search for COX1 in the [NUCLEOTIDE](#) database and then download the first five records.

Just COX1

```
mysearch = Entrez.esearch(db="nucleotide", term="COX1", retmax="5")
result1 = Entrez.read(mysearch)
```

Just CO1

```
mysearch = Entrez.esearch(db="nucleotide", term="CO1", retmax="5")
result2 = Entrez.read(mysearch)
```

Either COX1 or COI

```
mysearch = Entrez.esearch(db="nucleotide", term="COX1 or COI", retmax="5")
result3 = Entrez.read(mysearch)
```

What do they look like?

result1

```
{'Count': '1078946', 'RetMax': '5', 'RetStart': '0', 'IdList': ['3101929610', '3101906365', '3101906363', '3101874138', '3101874136'],
'TranslationSet': [], 'TranslationStack': [{'Term': 'COX1[All Fields]', 'Field': 'All Fields', 'Count': '1', 'Explode': 'N'}, 'GROUP'],
'QueryTranslation': 'COX1[All Fields]'}
```

What did we get back for the nucleotide search # 1?

```
for accession in result1['IdList']:
    genbank = Entrez.efetch(db="nucleotide", id=accession, rettype="gb")
    record = SeqIO.read(genbank, "genbank")
    print("COX1", record.description)
```

```
COX1 Eotetranychus sexmaculatus isolate Y1 cytochrome c oxidase subunit I (COX1) gene, par
COX1 Clinostomum piscidium isolate CP25 cytochrome c oxidase subunit I (COX1) gene, partia
COX1 Temnothorax ambiguus voucher Tambl cytochrome c oxidase subunit I (COX1) gene, partia
COX1 Oreochromis niloticus isolate ZSI/SRC F-1348 cytochrome c oxidase subunit I (COX1) ge
COX1 Oreochromis niloticus isolate ZSI/SRC FN1342 cytochrome c oxidase subunit I (COX1) ge
```

What did we get back for the nucleotide search # 2?

same for loop as above, just using result2 instead of result1

```
for accession in result2['IdList']:
    genbank = Entrez.efetch(db="nucleotide", id=accession, rettype="gb")
    record = SeqIO.read(genbank, "genbank")
    print("CO1", record.description)
```

```
CO1 Tachysurus ussuriensis isolate YJ_35
CO1 Tachysurus ussuriensis isolate YJ_34
CO1 Tachysurus ussuriensis isolate YJ_33
CO1 Tachysurus ussuriensis isolate YJ_32
CO1 Tachysurus ussuriensis isolate YJ_31
```

What did we get back for the nucleotide search # 3?

same for loop as above, just using result3 instead of result1

```
for accession in result3['IdList']:
    genbank = Entrez.efetch(db="nucleotide", id=accession, rettype="gb")
    record = SeqIO.read(genbank, "genbank")
```

<code>print("COX1 or COI",record.description)</code>	
COX1 or COI	Eotetranychus sexmaculatus isolate Y1 cytochrome c oxidase subunit I (COX1) ge
COX1 or COI	Clinostomum piscidium isolate CP25 cytochrome c oxidase subunit I (COX1) gene,
COX1 or COI	Temnothorax ambiguus voucher Tamb1 cytochrome c oxidase subunit I (COX1) gene,
COX1 or COI	Oreochromis niloticus isolate ZSI/SRC F-1348 cytochrome c oxidase subunit I (C
COX1 or COI	Oreochromis niloticus isolate ZSI/SRC FN1342 cytochrome c oxidase subunit I (C

Now let's do the searches again in the PROTEIN database and then download the first five records, combining the commands

```
result4 = Entrez.read(Entrez.esearch(db="protein", term="COX1", retmax="5"))
result5 = Entrez.read(Entrez.esearch(db="protein", term="CO1", retmax="5"))
result6 = Entrez.read(Entrez.esearch(db="protein", term="COX1 or COI", retmax="5"))
```

What do they look like?

result4

```
{'Count': '1177015', 'RetMax': '5', 'RetStart': '0', 'IdList': ['3101929611', '3101906366', '3101906364', '3101874139', '3101874137'],
'TranslationSet': [], 'TranslationStack': [{'Term': 'COX1[All Fields]', 'Field': 'All Fields', 'Count': '1177015', 'Explode': 'N'}, {'GROUP'}],
'QueryTranslation': 'COX1[All Fields]'}
```

What did we get back for the proteins search #4?

```
for accession in result4['IdList']:
    genbank = Entrez.efetch(db="protein", id=accession, rettype="gb")
    record = SeqIO.read(genbank, "genbank")
    print("COX1", record.description)
```

```
COX1 cytochrome c oxidase subunit I, partial (mitochondrion) [Eotetranychus sexmaculatus]
COX1 cytochrome c oxidase subunit I, partial (mitochondrion) [Clinostomum piscidium]
COX1 cytochrome c oxidase subunit I, partial (mitochondrion) [Temnothorax ambiguus]
COX1 cytochrome c oxidase subunit I, partial (mitochondrion) [Oreochromis niloticus]
COX1 cytochrome c oxidase subunit I, partial (mitochondrion) [Oreochromis niloticus]
```

What did we get back for the proteins search #5?

```
for accession in result5['IdList']:
    genbank = Entrez.efetch(db="protein", id=accession, rettype="gb")
    record = SeqIO.read(genbank, "genbank")
    print("CO1", record.description)
```

```
CO1 cytochrome c oxidase 1, partial (mitochondrion) [Tachysurus ussuriensis]
CO1 cytochrome c oxidase 1, partial (mitochondrion) [Tachysurus ussuriensis]
CO1 cytochrome c oxidase 1, partial (mitochondrion) [Tachysurus ussuriensis]
CO1 cytochrome c oxidase 1, partial (mitochondrion) [Tachysurus ussuriensis]
CO1 cytochrome c oxidase 1, partial (mitochondrion) [Tachysurus ussuriensis]
```

What did we get back for the proteins search #6?

```
for accession in result6['IdList']:
    genbank = Entrez.efetch(db="protein", id=accession, rettype="gb")
    record = SeqIO.read(genbank, "genbank")
    print("COX1 or COI", record.description)
```

```
COX1 or COI cytochrome c oxidase subunit I, partial (mitochondrion) [Eotetranychus sexmacu
COX1 or COI cytochrome c oxidase subunit I, partial (mitochondrion) [Clinostomum piscidium]
COX1 or COI cytochrome c oxidase subunit I, partial (mitochondrion) [Temnothorax ambiguus]
COX1 or COI cytochrome c oxidase subunit I, partial (mitochondrion) [Oreochromis niloticus]
COX1 or COI cytochrome c oxidase subunit I, partial (mitochondrion) [Oreochromis niloticus]
```

Something doesn't seem right?!

Be careful when entering the search term... `CO1` and `COI` might initially look the same, but they aren't...

Redo the nucleotide version for the CO1 ("number") and COI ("letter") gene

```
myresult_number_all = Entrez.read(Entrez.esearch(db="nucleotide", term="CO1", retmax="5"))
myresult_number_genes = Entrez.read(Entrez.esearch(db="nucleotide", term="CO1[Gene]", retm
myresult_letter_all = Entrez.read(Entrez.esearch(db="nucleotide", term="COI", retmax="5")
myresult_letter_genes = Entrez.read(Entrez.esearch(db="nucleotide", term="COI[Gene]", retm
```

How many did we find in the nucleotide database?

`myresult_number_all['Count']`

'137479'

`myresult_number_genes['Count']`

'84327'

`myresult_letter_all['Count']`

'3808538'

`myresult_letter_genes['Count']`

'3698953'

Redo the protein version for the CO1 ("number") and COI ("letter") gene

How many did we find in the protein database?

```
myresult_number_all = Entrez.read(Entrez.esearch(db="protein", term="CO1", retmax="5"))
myresult_number_genes = Entrez.read(Entrez.esearch(db="protein", term="CO1[Gene]", retmax=
myresult_letter_all = Entrez.read(Entrez.esearch(db="protein", term="COI", retmax="5"))
myresult_letter_genes = Entrez.read(Entrez.esearch(db="protein", term="COI[Gene]", retmax=
```

How many did we find in the protein database?

`myresult_number_all['Count']`

'93758'

`myresult_number_genes['Count']`

'61798'

`myresult_letter_all['Count']`

'3741840'

`myresult_letter_genes['Count']`

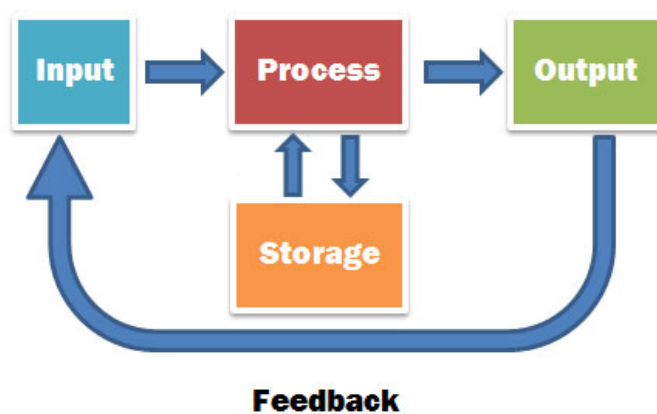
'3659203'

🏠 Exercises: your turn!

Make a directory within your Exercises space for today's work, and change directory to it.

```
mkdir ~/Exercises/Lecture20
```

```
cd ~/Exercises/Lecture20
```



1. 🏠 How many complete COX1 protein records are there for mammals?
 2. 🏠 What is their average length (the proteins that is, not the mammals!)?
 3. 🏠 Write a function that will answer the question for any gene name and any taxonomic group.
 4. 🏠 What else could you sensibly include in the function (in terms of retrieving possibly useful data)?
-

POSSIBLE ANSWERS

1. How many complete COX1 protein records are there for mammals?
2. What is their average length (the proteins that is, not the mammals!)?
3. Write a function that will answer the question for any gene name and any taxonomic group.
4. What else could you sensibly include in the function (in terms of retrieving possibly useful data)?

For now, let's assume that we can get the results we want by searching for

- "Mammalia COX1 complete"
- We will re-use the code from the lecture example to search
- We want the **protein** database, not the nucleotide one
- We will limit the results to just twenty records for now

```
# Programme to retrieve proteins from NCBI
```

```
# Written by s123456
```

```
# Version 1, 21 Nov 2025
```

```
# Import the required modules
```

```
from Bio import Entrez, SeqIO
```

```
import os, subprocess
```

```
Entrez.email = "s123456@ed.ac.uk"
```

```
Entrez.api_key = subprocess.check_output("echo ${NCBI_API_KEY}", shell=True).rstrip().decode()
```

```
# Define and do the search
```

```
result = Entrez.read(Entrez.esearch(db="protein", term="Mammalia COX1 complete", retmax="20"))
```

```
result
```

```
{'Count': '38398', 'RetMax': '20', 'RetStart': '0', 'IdList': ['2956674217', '2956674203', '2956674189', '2956674175', '2956674161', '2956674147', '2956674133', '2956674119', '2956674105', '2956674091', '2956674077', '2956674063', '2956674049', '2956674035', '2956674021', '2956674007', '2956673993', '2956673979', '2956673965', '2956673951'], 'TranslationSet': [{'From': 'Mammalia', 'To': 'Mammalia'[Organism] OR Mammalia[All Fields]}], 'TranslationStack': [{'Term': 'Mammalia'[Organism], 'Field': 'Organism', 'Count': '18604124', 'Explode': 'Y'}, {'Term': 'Mammalia[All Fields]', 'Field': 'All Fields', 'Count': '31276', 'Explode': 'N'}, {'OR', 'GROUP'}, {'Term': 'COX1[All Fields]', 'Field': 'All Fields', 'Count': '1177015', 'Explode': 'N'}, {'AND', {'Term': 'complete[All Fields]', 'Field': 'All Fields', 'Count': '92111655', 'Explode': 'N'}, {'AND', 'GROUP'}, 'QueryTranslation': '("Mammalia"[Organism] OR Mammalia[All Fields]) AND COX1[All Fields] AND complete[All Fields]}
```

```
# Extract info from the results of the search
```

```
count=1
```

```
for accession in result['IdList']:
```

```
    gb_file = Entrez.efetch(db="protein", id=accession, rettype="gb")
```

```
    record = SeqIO.read(gb_file, "genbank")
```

```
    print(count, record.id+"\t"+record.description+"\t"+record.seq)
```

```
    count += 1
```

```
    if count == 6 :
```

```
        break
```

1	XRJ24758.1	cytochrome c oxidase subunit I (mitochondrion)	[Homo sapiens]	MFADRWLFST
2	XRJ24745.1	cytochrome c oxidase subunit I (mitochondrion)	[Homo sapiens]	MFANRWLFST
3	XRJ24732.1	cytochrome c oxidase subunit I (mitochondrion)	[Homo sapiens]	MFADRWLFST
4	XRJ24719.1	cytochrome c oxidase subunit I (mitochondrion)	[Homo sapiens]	MFADRWLFST
5	XRJ24706.1	cytochrome c oxidase subunit I (mitochondrion)	[Homo sapiens]	MFADRWLFST

```
# What were the dictionary keys again?!
```

```
result.keys()
dict_keys(['Count', 'RetMax', 'RetStart', 'IdList', 'TranslationSet', 'TranslationStack', 'QueryTranslation'])

print("There were "+str(result['Count'])+" records found.")
There were 38398 records found.
```

Next, we want to get the length for each of the first five. The easiest way to do this is to get the sequence, then use `len(sequence)` to get the length, keeping a tally as we go so that we can work out the mean length.

```
# Programme to retrieve proteins from NCBI
# Written by s123456
# Version 2, 21 Nov 2025

# Import the required modules
from Bio import Entrez, SeqIO
import os, subprocess

Entrez.email = "s123456@ed.ac.uk"
Entrez.api_key=subprocess.check_output("echo ${NCBI_API_KEY}", shell=True).rstrip().decode()

# Define and do the search
result = Entrez.read(Entrez.esearch(db="protein", term="Mammalia COX1 complete", retmax="20"))

# Include a counter and a variable to hold the total length
loopcounter = total_length = 0

# Extract info from the results of the search,
# recalculating the mean length as we go
for accession in result['IdList']:
    loopcounter += 1
    gb_file = Entrez.efetch(db="protein", id=accession, rettype="gb")
    record = SeqIO.read(gb_file, "genbank")
    total_length = total_length + len(record.seq)
    mean_length = int(total_length/loopcounter)
    print(record.id+"\t"+record.description+"\t"+str(len(record.seq))+"\t"+record.seq[0:10])

XRJ24758.1      cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens] 513 MF.
XRJ24745.1      cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens] 513 MF.
XRJ24732.1      cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens] 513 MF.
XRJ24719.1      cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens] 513 MF.
XRJ24706.1      cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens] 513 MF.
XRJ24693.1      cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens] 513 MF.
XRJ24680.1      cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens] 513 MF.
XRJ24667.1      cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens] 513 MF.
XRJ24654.1      cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens] 513 MF.
XRJ24641.1      cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens] 513 MF.
XRJ24628.1      cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens] 513 MF.
XRJ24615.1      cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens] 513 MF.
XRJ24602.1      cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens] 513 MF.
XRJ24589.1      cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens] 513 MF.
XRJ24576.1      cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens] 513 MF.
XRJ24563.1      cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens] 513 MF.
XRJ24550.1      cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens] 513 MF.
XRJ24537.1      cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens] 513 MF.
XRJ24524.1      cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens] 513 MF.
XRJ24511.1      cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens] 513 MF.

# Output some useful info
```

```
print("There were "+str(result['Count'])+" records found.")
There were 38398 records found.
print("The mean length was "+str(mean_length)+" amino acids.")
The mean length was 513 amino acids.
```

We can now turn this code into a function that takes a taxonomic name and a gene name and returns the average length of the first x number (user-defined) results; we'll also keep the outputs and write the info out to a file too.

```
# Programme to retrieve proteins from NCBI
# Written by s123456
# Version 3, 21 Nov 2025
#

# Set up a list object to hold the outputs all the session's searches
search_results = []

# Function with three arguments: taxonomic group (text, not taxid), gene name, number to return
# NOTE_incomplete: could add another argument to cater for DNA or protein database search?
# NOTE_incomplete: could add another argument to cater for user email address?
def get_average_length(taxonomy, gene, howmany=10):
    from Bio import Entrez, SeqIO
    import os, subprocess
    Entrez.email = "s123456@ed.ac.uk"
    Entrez.api_key=subprocess.check_output("echo ${NCBI_API_KEY}", shell=True).rstrip().decode()
    search_term = taxonomy + " " + gene + " complete"

# Set up output file, named after search used (spaces removed!)
    search_output = open(search_term.replace(" ", "_")+"_outputs.txt","w")
    mysearch = Entrez.esearch(db="protein", term=search_term, retmax=howmany)
    result = Entrez.read(mysearch)
    loopcounter = total_length = 0

# Extract info from the results of the search
    for accession in result['IdList']:
        loopcounter += 1
        gb_file = Entrez.efetch(db="protein", id=accession, rettype="gb")
        record = SeqIO.read(gb_file, "genbank")
        total_length = total_length + len(record.seq)

# Add the results of the search to search_results
    search_results.append([search_term, record.id, record.description, len(record.seq), record.description])

# Show the results (trimmed sequence) of the search to screen as we go
    print(record.id+"\t"+record.description+"\t"+str(len(record.seq))+"\t"+record.seq[0:100])

# Write results to search output file
    search_output.write(record.id+"\t"+record.description+"\t"+str(len(record.seq))+"\t"+record.seq[0:100])

# Interim print statment for mean length
    mean_length = int(total_length/loopcounter)
    return print("\nThe mean length was "+str(mean_length)+" amino acids.\n")
    close(search_output)
```

What is **wrong with the above** function?! Have a I broken an encapsulation rule, and if so, what have I done wrong...?

Let's try it anyway...

```
get_average_length('Nematoda', 'COX1')
```

```
XJP06356.1    cytochrome c oxidase subunit I, partial (mitochondrion) [Seuratascaris sp. o LL-2024]    128    ILILPAFGIIIS
XJP06355.1    cytochrome c oxidase subunit I, partial (mitochondrion) [Seuratascaris sp. o LL-2024]    128    ILILPAFGIIISQ
A8Y2C9.1      RecName: Full= SURF1-like protein    317    MAFRWAIQMALRRGGGGQRLLLTHQHQQHONYQFFKTSSTQFSTRNSQIL...
Q9N5N8.1      RecName: Full= SURF1-like protein    323    MFLPAAAFARILVVSQRQLFNTRIIHTSGSGYSITQCRFYKTTPLCTHKQ...
P24893.2      RecName: Full= Cytochrome c oxidase subunit 1; AltName: Full= Cytochrome c oxidase polypeptide I    525    MNLYKKYQ
XUB15251.1    cytochrome c oxidase subunit I (mitochondrion) [Rhabdias macrocephalum]    524    MSVNSYNKISWKMMWLESSNNHKDIGTLY
WXT83216.1    cytochrome c oxidase subunit I, partial (mitochondrion) [Rhabdias macrocephalum]    218    TLYFIFGLWSGM
WXT83215.1    cytochrome c oxidase subunit I, partial (mitochondrion) [Rhabdias macrocephalum]    218    TLYFIFGLWSGM
WXT83214.1    cytochrome c oxidase subunit I, partial (mitochondrion) [Rhabdias macrocephalum]    218    TLYFIFGLWSGM
XRT72422.1    cytochrome c oxidase subunit 1 (mitochondrion) [Strongyloides fuelleborni fuelleborni]    551    MCEELNGLYTWC
```

The mean length was 315 amino acids.

get_average_length('Mammalia', 'COX1')

```
XRJ24758.1    cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens]    513    MFADRWLFTSTNHKDIGTLYLLFGAWAGVLGTALSLL
XRJ24745.1    cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens]    513    MFANRWLFTSTNHKDIGTLYLLFGAWAGVLGTALSLL
XRJ24732.1    cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens]    513    MFADRWLFTSTNHKDIGTLYLLFGAWAGVLGTALSLL
XRJ24719.1    cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens]    513    MFADRWLFTSTNHKDIGTLYLLFGAWAGVLGTALSLL
XRJ24706.1    cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens]    513    MFADRWLFTSTNHKDIGTLYLLFGAWAGVLGTALSLL
XRJ24693.1    cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens]    513    MFADRWLFTSTNHKDIGTLYLLFGAWAGVLGTALSLL
XRJ24680.1    cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens]    513    MFADRWLFTSTNHKDIGTLYLLFGAWAGVLGTALSLL
XRJ24667.1    cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens]    513    MFANRWLFTSTNHKDIGTLYLLFGAWAGVLGTALSLL
XRJ24654.1    cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens]    513    MFADRWLFTSTNHKDIGTLYLLFGAWAGVLGTALSLL
XRJ24641.1    cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens]    513    MFADRWLFTSTNHKDIGTLYLLFGAWAGVLGTALSLL
```

The mean length was 513 amino acids.

get_average_length('Arthropoda', 'ATP6')

```
ART64936.1    ATP synthase F0 subunit 6 (mitochondrion) [Parnassius actius]    225    MMSNLFISFDPTTNFLNLSLNWIKTLIGLMFPIYSF
BBF67790.1    ATP synthase F0 subunit 6 (mitochondrion) [Bombus florilegus]    225    MKMIMMNLFTFDPSINYFQQLNWIIFMLTPIIFPN
KKH90628.1    ATP synthase F0 subunit 6 (mitochondrion) [Heterotrigona itama]    227    MVILTSLFESFDPSVYMYMFKFQLNWMFVLVPIIFLC
KKH90619.1    ATP synthase F0 subunit 6 (mitochondrion) [Heterotrigona itama]    227    MVILTSLFESFDPSVYMYMFKFQLNWMFVLVPIIFLC
KKH90605.1    ATP synthase F0 subunit 6 (mitochondrion) [Heterotrigona itama]    227    MVILTSLFESFDPSVYMYMFKFQLNWMFVLVPIIFLC
KKH90596.1    ATP synthase F0 subunit 6 (mitochondrion) [Heterotrigona itama]    227    MVILTSLFESFDPSVYMYMFKFQLNWMFVLVPIIFLC
YAS66118.1    ATP synthase F0 subunit 6 (mitochondrion) [Anomala sp. T73439-1]    215    MMTNLFSVFDPC7GMLYLNLWLTSTFFIFII
YAR15166.1    ATP synthase F0 subunit 6 (mitochondrion) [Tropilaelaps mercedesae]    221    MLTNIFISFDPTSSLSFSSNNWLSMFLSF
YAR15071.1    ATP synthase F0 subunit 6 (mitochondrion) [Nicrophorus quadraticollis]    223    MMTNLFSFDPTSSLSFSSNNWLSMFLSF
XLT11369.1    ATP synthase F0 subunit 6 (mitochondrion) [Pontastacus leptodactylus]    224    MVASLFSIFDPSSSIFISISLNWVSTFLG
```

The mean length was 224 amino acids.

What does search_results contain?

```
for finds in search_results :
    print(finds)
```

```
['Nematoda COX1 complete', 'XJP06356.1', 'cytochrome c oxidase subunit I, partial (mitochondrion) [Seuratascaris sp. o LL-2024]', 128, Seq('ILILPAFGIIISQSSLYLTGQEVFALGAVYALLSIGLIGCVVAHMYTVGMD...TTY')]
['Nematoda COX1 complete', 'XJP06355.1', 'cytochrome c oxidase subunit I, partial (mitochondrion) [Seuratascaris sp. o LL-2024]', 128, Seq('ILILPAFGIIISQSSLYLTGQEVFALGAVYALLSIGLIGCVVAHMYTVGMD...TTY')]
['Nematoda COX1 complete', 'Q9N5N8.1', 'RecName: Full= SURF1-like protein', 317, Seq('MAFRWAIQMALRRGGGGQRLLLTHQHQQHONYQFFKTSSTQFSTRNSQIL...FRF')]
['Nematoda COX1 complete', 'P24893.2', 'RecName: Full= Cytochrome c oxidase subunit 1; AltName: Full= Cytochrome c oxidase polypeptide I', 525, Seq('MNLYKKYQGLAWLWLESSNNHKDIGTLYLTFGLWSGMQVFTSLLIRLAKPGF...LNN')]
['Nematoda COX1 complete', 'XUB15251.1', 'cytochrome c oxidase subunit I (mitochondrion) [Rhabdias macrocephalum]', 524, Seq('MSVNSYNKISWKMMWLESSNNHKDIGTLYLTFGLWSGMQVFTSLLIRLAKPGF...RDV')]
['Nematoda COX1 complete', 'WXT83216.1', 'cytochrome c oxidase subunit I, partial (mitochondrion) [Rhabdias macrocephalum]', 218, Seq('TLYFIFGLWSGMQVFTSLLIRLAKPGFPMRQGLNYSITANALIMTFPMV...HLF')]
['Nematoda COX1 complete', 'WXT83215.1', 'cytochrome c oxidase subunit I, partial (mitochondrion) [Rhabdias macrocephalum]', 218, Seq('TLYFIFGLWSGMQVFTSLLIRLAKPGFPMRQGLNYSITANALIMTFPMV...HLF')]
['Nematoda COX1 complete', 'WXT83214.1', 'cytochrome c oxidase subunit I, partial (mitochondrion) [Rhabdias macrocephalum]', 218, Seq('TLYFIFGLWSGMQVFTSLLIRLAKPGFPMRQGLNYSITANALIMTFPMV...HLF')]
['Mammalia COX1 complete', 'XRJ24758.1', 'cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens]', 513, Seq('MFADRWLFTSTNHKDIGTLYLLFGAWAGVLGTALSLLIRLAKPGQPNLLGDEHY...MS')]
['Mammalia COX1 complete', 'XRJ24745.1', 'cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens]', 513, Seq('MFADRWLFTSTNHKDIGTLYLLFGAWAGVLGTALSLLIRLAKPGQPNLLGDEHY...MS')]
['Mammalia COX1 complete', 'XRJ24732.1', 'cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens]', 513, Seq('MFADRWLFTSTNHKDIGTLYLLFGAWAGVLGTALSLLIRLAKPGQPNLLGDEHY...MS')]
['Mammalia COX1 complete', 'XRJ24719.1', 'cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens]', 513, Seq('MFADRWLFTSTNHKDIGTLYLLFGAWAGVLGTALSLLIRLAKPGQPNLLGDEHY...MS')]
['Mammalia COX1 complete', 'XRJ24706.1', 'cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens]', 513, Seq('MFADRWLFTSTNHKDIGTLYLLFGAWAGVLGTALSLLIRLAKPGQPNLLGDEHY...MS')]
['Mammalia COX1 complete', 'XRJ24693.1', 'cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens]', 513, Seq('MFADRWLFTSTNHKDIGTLYLLFGAWAGVLGTALSLLIRLAKPGQPNLLGDEHY...MS')]
['Mammalia COX1 complete', 'XRJ24680.1', 'cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens]', 513, Seq('MFADRWLFTSTNHKDIGTLYLLFGAWAGVLGTALSLLIRLAKPGQPNLLGDEHY...MS')]
['Mammalia COX1 complete', 'XRJ24667.1', 'cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens]', 513, Seq('MFADRWLFTSTNHKDIGTLYLLFGAWAGVLGTALSLLIRLAKPGQPNLLGDEHY...MS')]
['Mammalia COX1 complete', 'XRJ24654.1', 'cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens]', 513, Seq('MFADRWLFTSTNHKDIGTLYLLFGAWAGVLGTALSLLIRLAKPGQPNLLGDEHY...MS')]
['Mammalia COX1 complete', 'XRJ24641.1', 'cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens]', 513, Seq('MFADRWLFTSTNHKDIGTLYLLFGAWAGVLGTALSLLIRLAKPGQPNLLGDEHY...MS')]
['Arthropoda ATP6 complete', 'ART64936.1', 'ATP synthase F0 subunit 6 (mitochondrion) [Parnassius actius]', 225, Seq('MMSNLFISFDPTTNFLNLSLNWIKTLIGLMFPIYSF...RWN')]
['Arthropoda ATP6 complete', 'BBF67790.1', 'ATP synthase F0 subunit 6 (mitochondrion) [Bombus florilegus]', 225, Seq('MKMIMMNLFTFDPSINYFQQLNWIIFMLTPIIFPN...ESN')]
['Arthropoda ATP6 complete', 'KKH90628.1', 'ATP synthase F0 subunit 6 (mitochondrion) [Heterotrigona itama]', 227, Seq('MVILTSLFESFDPSVYMYMFKFQLNWMFVLVPIIFLCNYSWTFPSRMDCHLL...ESR')]
['Arthropoda ATP6 complete', 'KKH90619.1', 'ATP synthase F0 subunit 6 (mitochondrion) [Heterotrigona itama]', 227, Seq('MVILTSLFESFDPSVYMYMFKFQLNWMFVLVPIIFLCNYSWTFPSRMDCHLL...ESR')]
['Arthropoda ATP6 complete', 'KKH90605.1', 'ATP synthase F0 subunit 6 (mitochondrion) [Heterotrigona itama]', 227, Seq('MVILTSLFESFDPSVYMYMFKFQLNWMFVLVPIIFLCNYSWTFPSRMDCHLL...ESR')]
['Arthropoda ATP6 complete', 'KKH90596.1', 'ATP synthase F0 subunit 6 (mitochondrion) [Heterotrigona itama]', 227, Seq('MVILTSLFESFDPSVYMYMFKFQLNWMFVLVPIIFLCNYSWTFPSRMDCHLL...ESR')]
['Arthropoda ATP6 complete', 'YAS66118.1', 'ATP synthase F0 subunit 6 (mitochondrion) [Anomala sp. T73439-1]', 215, Seq('MMTNLFISVFDPC7GMLYLNLWLTSTFFIFII...RWN')]
['Arthropoda ATP6 complete', 'YAR15166.1', 'ATP synthase F0 subunit 6 (mitochondrion) [Tropilaelaps mercedesae]', 221, Seq('MLTNIFISFDPTSSLSFSSNNWLSMFLSF...NEI')]
['Arthropoda ATP6 complete', 'YAR15071.1', 'ATP synthase F0 subunit 6 (mitochondrion) [Nicrophorus quadraticollis]', 223, Seq('MTNLFSFDPTSSLSFSSNNWLSMFLSF...RWN')]
['Arthropoda ATP6 complete', 'XLT11369.1', 'ATP synthase F0 subunit 6 (mitochondrion) [Pontastacus leptodactylus]', 224, Seq('MVASLFSIFDPSSSIFISISLNWVSTFLG...RWN')]
```

How about the files we made (1 for each get_average_length() search)?

```
import os, glob
```

```
os.listdir()
```

```
['Mammalia_COX1_complete_outputs.txt', 'Nematoda_COX1_complete_outputs.txt',
'Arthropoda_ATP6_complete_outputs.txt']
```

```
sorted(glob.glob("*outputs.txt"))
```

```
['Arthropoda_ATP6_complete_outputs.txt', 'Mammalia_COX1_complete_outputs.txt',
'Nematoda_COX1_complete_outputs.txt']
```

Let have a quick look at one of them!

```
print(open("Nematoda_COX1_complete_outputs.txt").read().strip("\n"))
```

```
XJP06356.1      cytochrome c oxidase subunit I, partial (mitochondrion) [Seuratascaris sp.
XJP06355.1      cytochrome c oxidase subunit I, partial (mitochondrion) [Seuratascaris sp.
A8Y2C9.1        RecName: Full=SURF1-like protein          317      MAFRWAIQMALRRGGGGGQRLLLTQH
Q9N5N8.1        RecName: Full=SURF1-like protein          323      MPLPAAAFARILVVSQRQLFNTRIHH
P24893.2        RecName: Full=Cytochrome c oxidase subunit 1; AltName: Full=Cytochrome c o
XUB15251.1      cytochrome c oxidase subunit I (mitochondrion) [Rhabdias macrocephalum] 52
WXT83216.1      cytochrome c oxidase subunit I, partial (mitochondrion) [Rhabdias macrocep
WXT83215.1      cytochrome c oxidase subunit I, partial (mitochondrion) [Rhabdias macrocep
WXT83214.1      cytochrome c oxidase subunit I, partial (mitochondrion) [Rhabdias macrocep
XRT72422.1      cytochrome c oxidase subunit 1 (mitochondrion) [Strongyloides fuelleborni
...
```

```
# Sometimes protein sequences aren't high quality (have ambiguous amino acids...)
```

```
AYI68512.1      cytochrome c oxidase subunit I (mitochondrion) [Bos taurus].
...
MFINRWLFSTNHKDIGTLYLLFGAWAGMVGTA LSLXXXXXXXXXXXXXXXXXXXX QIYNVVVTAHAFVMIFFMVMPIMIGGFGNWLVP...
```

```
idwanted=Entrez.read(Entrez.esearch(db="protein", term="AYI68512", retmax=10))['IdList']
```

```
idwanted
```

```
['1489705257']
```

```
print(SeqIO.read(Entrez.efetch(db="protein", id=idwanted, rettype="fasta"), "fasta").seq)
```

```
MFINRWLFSTNHKDIGTLYLLFGAWAGMVGTA LSLXXXXXXXXXXXXXXXXXXXX QIYNVVVTAHAFVMIFFMVMPIMIGGFGNWLVP L MIGA
RMNNSFWLLPPSFLLLLASSMVEAGAGTGWTVPPLAGNLAHAGASVDLTIFSLHLAGVSSILGAINFITTIINMKPPAMSQYQTPLFV
TAVLLLLSLPVLAAGITMLLTDRNLNTTFFDPAGGGDPILYQHLFWFFGHPEVYILILPGFGMISHIVTYYSKGKEPFGYMGVMWAMMSI
IVWAHMFVGMVDVTRAYFTSATMIIAIP TGVKVFSWLATLHG GNIKWSPAMMWALGFIFLFTVGGLTGIVLANS SLDIVLHDTYYVVA
LSMGAVFAIMGGFVHWFP L FSGYTLNDTWAKIHFAXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX MWNTISSMGSFISLTAVMLMV
FASKREVLTVDLTTTNLEWLN GCPPPYHTFEEPTYVNLK
```

```
SeqIO.read(Entrez.efetch(db="protein", id=idwanted, rettype="fasta"), "fasta").seq.count('X')
```

```
49
```

```
# Programme to retrieve proteins from NCBI
```

```
# Written by s123456
```

```
# Version 4, 21 Nov 2025
```

```
#
```

```
# Dealing with X amino acids
```

```
search_results = []
```

```
# Improved function?
```

```
def get_average_length2(taxonomy, gene, startat=0, howmany=100):
    from Bio import Entrez, SeqIO
    Entrez.email = "s123456@ed.ac.uk"
    Entrez.api_key = subprocess.check_output("echo ${NCBI_API_KEY}", shell=True).rstrip().de
    search_term = taxonomy + " " + gene + " complete"
    search_output = open(search_term.replace(" ", "_") + "_outputs.txt", "w")
    mysearch = Entrez.esearch(db="protein", term=search_term, retstart = startat, retmax=h
    result = Entrez.read(mysearch)
    print("Search done,", result['Count'], "found, starting retrieval of", howmany, "starting
    loopcounter = poorseqs = total_length = 0
    for accession in result['IdList']:
        loopcounter += 1
        genbank = Entrez.efetch(db="protein", id=accession, rettype="gb")
        print('\r' + 'Retrieving sequence ' + str(loopcounter+startat), end="")
        record = SeqIO.read(genbank, "genbank")
        Xaa = str(record.seq).count("X")
        if Xaa > 5 :
            print(" Seq", loopcounter+startat, "contains", Xaa, "unknown amino acids:", int(100*Xa
            poorseqs += 1
        else :
            total_length = total_length + len(record.seq)
```

```
search_results.append([search_term,record.id,record.description,len(record.seq),r
# print(record.id+"\t"+record.description+"\t"+str(len(record.seq))+ "\t"+record.s
search_output.write(record.id+"\t"+record.description+"\t"+str(len(record.seq))+ "\t"
goodseqs=loopcounter-poorseqs
mean_length = int(total_length/goodseqs)
return print("\n\nThe mean length of the "+str(goodseqs)+" high quality seqs was "+str(
close(search_output)
```

Run it to check (retrieval is VERY slow), just a subset here...

get_average_length2('Bos taurus', 'cytochrome c oxidase subunit I (mitochondrion)',1,250)

Search done, 3102 found, starting retrieval of 250 starting at 1

...

```
Retrieving sequence 60 Seq 60 contains 166 unknown amino acids: 32 percent of total!
Retrieving sequence 61 Seq 61 contains 58 unknown amino acids: 22 percent of total!
Retrieving sequence 64 Seq 64 contains 54 unknown amino acids: 20 percent of total!
Retrieving sequence 65 Seq 65 contains 73 unknown amino acids: 32 percent of total!
Retrieving sequence 66 Seq 66 contains 71 unknown amino acids: 13 percent of total!
Retrieving sequence 67 Seq 67 contains 8 unknown amino acids: 3 percent of total!
Retrieving sequence 68 Seq 68 contains 11 unknown amino acids: 4 percent of total!
Retrieving sequence 69 Seq 69 contains 74 unknown amino acids: 14 percent of total!
Retrieving sequence 70 Seq 70 contains 260 unknown amino acids: 50 percent of total!
Retrieving sequence 85 Seq 85 contains 66 unknown amino acids: 25 percent of total!
Retrieving sequence 86 Seq 86 contains 102 unknown amino acids: 44 percent of total!
Retrieving sequence 87 Seq 87 contains 201 unknown amino acids: 39 percent of total!
Retrieving sequence 251
```

The mean length of the 238 high quality seqs was 301 amino acids.

Bulk downloading of fasta sequences

search_term="Mammalia COX1"

mysearch = Entrez.esearch(db="protein", term=search_term, retmax=150000)

result = Entrez.read(mysearch)

How many are there?

result['Count']

'108235'

len(result['IdList'])

108235

all_fastas = open("all_fastas.faa", "w")

all_fastas.write(Entrez.efetch(db="protein", id=result['IdList'], retmode="text", rettype="fasta").read())

5030652

all_fastas.close()

open("all_fastas.faa").read().count(">")

9999

Huh?! That took ages , and we were limited to the first 9999!

Could use retstart=somenumber and retmax=10000 (how many to return)

See [NCBI help](#)

OR we can use result['IdList'] to make sets of 10000 to download!

result['IdList'][0:3]

['3071768259', '3071768257', '3071768255']

batch_size = 10000

list(range(0,int(result['Count']),batch_size))

```
[0, 10000, 20000, 30000, 40000, 50000, 60000, 70000, 80000, 90000, 100000, 110000]
```

```
batch_starts=list(range(0,int(result['Count']),batch_size))
```

```
# Let's see how long this is taking: need the datetime module...
```

```
import datetime
```

```
# Create file connection for saving output
```

```
all_fastas = open("all_fastas.faa","w")
```

```
# for loop for efetching
```

```
for from_here in batch_starts :  
    print(datetime.datetime.now().strftime('## %d %b %Y, %H:%M:%S'))  
    to_here = from_here + batch_size  
    print("Getting the sequences",from_here+1,"to",to_here)  
    all_fastas.write(Entrez.efetch(db="protein", \  
    id=result['IdList'][from_here:to_here], \  
    retmode="text", rettype="fasta").read())
```

```
print("\n",datetime.datetime.now().strftime('## %d %b %Y, %H:%M:%S'))
```

```
## 13 Nov 2025, 14:17:15  
Getting the sequences 1 to 10000  
5030652  
## 13 Nov 2025, 14:18:10  
Getting the sequences 10001 to 20000  
5106488  
## 13 Nov 2025, 14:19:08  
Getting the sequences 20001 to 30000  
4970489  
## 13 Nov 2025, 14:20:22  
Getting the sequences 30001 to 40000  
5272431  
## 13 Nov 2025, 14:21:26  
Getting the sequences 40001 to 50000  
5813205  
## 13 Nov 2025, 14:22:26  
Getting the sequences 50001 to 60000  
5907265  
## 13 Nov 2025, 14:23:25  
Getting the sequences 60001 to 70000  
5930759  
## 13 Nov 2025, 14:28:04  
Getting the sequences 70001 to 80000  
5617673  
## 13 Nov 2025, 14:29:46  
Getting the sequences 80001 to 90000  
5822418  
## 13 Nov 2025, 14:30:48  
Getting the sequences 90001 to 100000  
5926873  
## 13 Nov 2025, 14:32:09  
Getting the sequences 100001 to 110000  
4888091  
## 13 Nov 2025, 14:33:51
```

```
# Close file connection for output
```

```
all_fastas.close()
```

```
# Check output
```

```
open("all_fastas.faa").read().count(">")
```

```
108225
```

```
os.system("grep -c \">\" all_fastas.faa")
```


108225
0

A wee function to get a range of IDs out

```
def esearch_process(stuff=result, start=0, stop=0) :  
    gotfasta=[]  
    print("Processing from",start,"to",stop,"...")  
    gotfasta.append(Entrez.efetch(db="protein", id=stuff['IdList'][start:stop], retmode="t"  
    return gotfasta
```

len(result['IdList'])

108225

```
print("\n".join(esearch_process(start=99999,stop=100003)))
```

Processing from 99999 to 100003 ...

```
>AGW56324.1 cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens]  
MFADRWLFSTNHKDIGTLYLLFGAWAGVLGTALSLLIRAEQGPGNLLGNDHIYNVIVTAHAFVMIFFMV  
MPIMIGGGFNGWLVPMLIGAPDMAFPRMNNMSFWLLPPSLLLLLASAMVEAGAGTGWTVYPPLAGNYSHPG  
ASVDLTIFSLHLAGVSSILGAINFITTIINMKPPAMTQYQTPLFVWSVLITAVLLLLSLPVLAAAGITMLL  
TDRNLNTTFFDPAGGGDPILYQHLFWFFGHPEVYIILPGFGMISHIVTYYSKGKEPFGYMGVMWAMMSI  
GFLGFIVWAHMHMFTVGMDDVDTRAYFTSATMIIAIP TGKVFVSWLATLHGSNMKWSAAVLWALGFIFLFTV  
GGLTGIVLANSSLDIVLHDTYYVVAHFHYVLSMGAVFAIMGGFIHWFPLFSGYTLDTYAKIHFTIMFIG  
VNLTFFPQHFLGLSGMPRRYSYDPDAYTTWNILSSVGSFISLTAVMLMIFMIWEAFASKRKVLMVEEPSM  
NLEWLYGCPPPYHTFEFPVYMK
```

```
>AGW56311.1 cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens]  
MFADRWLFSTNHKDIGTLYLLFGAWAGVLGTALSLLIRAEQGPGNLLGNDHIYNVIVTAHAFVMIFFMV  
MPIMIGGGFNGWLVPMLIGAPDMAFPRMNNMSFWLLPPSLLLLLASAMVEAGAGTGWTVYPPLAGNYSHPG  
ASVDLTIFSLHLAGVSSILGAINFITTIINMKPPAMTQYQTPLFVWSVLITAVLLLLSLPVLAAAGITMLL  
TDRNLNTTFFDPAGGGDPILYQHLFWFFGHPEVYIILPGFGMISHIVTYYSKGKEPFGYMGVMWAMMSI  
GFLGFIVWAHMHMFTVGMDDVDTRAYFTSATMIIAIP TGKVFVSWLATLHGSNMKWSAAVLWALGFIFLFTV  
GGLTGIVLANSSLDIVLHDTYYVVAHFHYVLSMGAVFAIMGGFIHWFPLFSGYTLDTYAKIHFTIMFIG  
VNLTFFPQHFLGLSGMPRRYSYDPDAYTTWNILSSVGSFISLTAVMLMIFMIWEAFASKRKVLMVEEPSM  
NLEWLYGCPPPYHTFEFPVYMK
```

```
>AGW56298.1 cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens]  
MFADRWLFSTNHKDIGTLYLLFGAWAGVLGTALSLLIRAEQGPGNLLGNDHIYNVIVTAHAFVMIFFMV  
MPIMIGGGFNGWLVPMLIGAPDMAFPRMNNMSFWLLPPSLLLLLASAMVEAGAGTGWTVYPPLAGNYSHPG  
ASVDLTIFSLHLAGVSSILGAINFITTIINMKPPAMTQYQTPLFVWSVLITAVLLLLSLPVLAAAGITMLL  
TDRNLNTTFFDPAGGGDPILYQHLFWFFGHPEVYIILPGFGMISHIVTYYSKGKEPFGYMGVMWAMMSI  
GFLGFIVWAHMHMFTVGMDDVDTRAYFTSATMIIAIP TGKVFVSWLATLHGSNMKWSAAVLWALGFIFLFTV  
GGLTGIVLANSSLDIVLHDTYYVVAHFHYVLSMGAVFAIMGGFIHWFPLFSGYTLDTYAKIHFTIMFIG  
VNLTFFPQHFLGLSGMPRRYSYDPDAYTTWNILSSVGSFISLTAVMLMIFMIWEAFASKRKVLMVEEPSM  
NLEWLYGCPPPYHTFEFPVYMK
```

```
>AGW56285.1 cytochrome c oxidase subunit I (mitochondrion) [Homo sapiens]  
MFADRWLFSTNHKDIGTLYLLFGAWAGVLGTALSLLIRAEQGPGNLLGNDHIYNVIVTAHAFVMIFFMV  
MPIMIGGGFNGWLVPMLIGAPDMAFPRMNNMSFWLLPPSLLLLLASAMVEAGAGTGWTVYPPLAGNYSHPG  
ASVDLTIFSLHLAGVSSILGAINFITTIINMKPPAMTQYQTPLFVWSVLITAVLLLLSLPVLAAAGITMLL  
TDRNLNTTFFDPAGGGDPILYQHLFWFFGHPEVYIILPGFGMISHIVTYYSKGKEPFGYMGVMWAMMSI  
GFLGFIVWAHMHMFTVGMDDVDTRAYFTSATMIIAIP TGKVFVSWLATLHGSNMKWSAAVLWALGFIFLFTV  
GGLTGIVLANSSLDIVLHDTYYVVAHFHYVLSMGAVFAIMGGFIHWFPLFSGYTLDTYAKIHFTIMFIG  
VNLTFFPQHFLGLSGMPRRYSYDPDAYTTWNILSSVGSFISLTAVMLMIFMIWEAFASKRKVLMVEEPSM  
NLEWLYGCPPPYHTFEFPVYMK
```

We would probably want to consider doing things in parallel perhaps? In the above situation, the time is mostly spent getting the data from NCBI, so we are at the mercy of how busy their server is, as well as how good our internet connection is.

If you are interested in exploring how to run processes in parallel, do have a look at these links. The third one illustrates very well the variety of ways we might want to do a task, depending on the task... guess what, there is more than one way of doing things...!


```
import multiprocessing as mp
```

```
print("Number of processors: ", mp.cpu_count())
```

```
Number of processors: 256
```

- [Link 1: Machine Learning Plus](#)
- [Link 2: Sitepoint](#)
- [Link 3: StackOverflow](#), good examples starting at the heading "**This IS the easiest way to do it!**"

For **LOTS** more BioPython, DO check out the [BioPython tutorial](#) mentioned at the beginning of today's lecture: you might well need it in your project!

🏠git and GitHub time!

Local repository actions (the absolute minimum set of commands required here...)

git st

On branch master

Changes not staged for commit:

```
(use "git add/rm <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
    deleted:    ../Lecture04/CodeFiles.tar.gz
    modified:   ../Lecture06/run_this_script_v1
```

Untracked files:

```
(use "git add <file>..." to include in what will be committed)
    ../BN.out
    ../BN.sh
    ../Lecture04/Als_ICA/
    ../Lecture04/all_the_things_I_did
    ../Lecture04/outs/
    ../Lecture05/Country.Mozambique.details
    ../Lecture07/Cosmoscarta.nuc.fa
    ../Lecture07/Cosmoscarta.nuc.gis
    ../Lecture07/myfileofaccessions.txt
    ../Lecture12/AJ223353.fasta
    ../Lecture12/AJ223353.genbank
    ../Lecture12/AJ223353_noheader.fasta
    ../Lecture12/AJ223353_noheader.fasta2
    ../Lecture12/AJ223353_noheader.fasta3
    ../Lecture12/All_exons.fasta
    ../Lecture12/All_noncodings.fasta
    ../Lecture12/local_exon01.fasta
    ../Lecture12/local_exon02.fasta
    ../Lecture12/local_intron01.fasta
    ../Lecture12/my_dna.txt
    ../Lecture12/plain_genomic_seq.txt
    ../Lecture12/quick_blast_check.out
    ../Lecture12/remote_exon01.fasta
    ../Lecture12/remote_noncoding01.fasta
    ../Lecture12/remote_noncoding02.fasta
    ../Lecture12/shutilversion.txt
    ../Lecture13/100_199/
    ../Lecture13/200_299/
    ../Lecture13/300_399/
    ../Lecture13/400_499/
    ../Lecture13/500_599/
    ../Lecture13/600_699/
    ../Lecture13/700_799/
    ../Lecture13/800_899/
    ../Lecture13/900_999/
    ../Lecture13/Cleaned_sequences.txt
```

```
../Lecture13/Cleaned_sequences2.txt
../Lecture13/Cleaned_sequences3.txt
../Lecture13/Exercise2_coding_sequence.fasta
../Lecture13/Lecture13.tar.gz
../Lecture13/dna_files/
../Lecture13/exons.txt
../Lecture13/genomic_dna2.txt
../Lecture13/input.txt
../Lecture13/remote_exon01.fasta
../Lecture14/__pycache__/
../Lecture14/data.csv
../Lecture16/__pycache__/
../Lecture17/All_the_Arabidopsis_ones.tsv
../Lecture17/eukaryotes.txt
../Lecture17/playtime/
../Lecture19/Chart_01.png
../Lecture19/Chart_02.png
../Lecture19/Chart_03.png
../Lecture19/Chart_04.png
../Lecture19/Chart_05.png
../Lecture19/Chart_06.png
../Lecture19/Chart_07.png
../Lecture19/Chart_08.png
../Lecture19/Chart_09.png
../Lecture19/Chart_10.png
../Lecture19/Chart_11.png
../Lecture19/Chart_12.png
../Lecture19/Chart_13.png
../Lecture19/Chart_14.png
../Lecture19/Chart_15.png
../Lecture19/Chart_16.png
../Lecture19/Chart_16A.png
../Lecture19/Chart_77.png
./
```

no changes added to commit (use "git add" and/or "git commit -a")

What files did we generate?

ls -l

```
all_fastas.faa
Arthropoda_ATP6_complete_outputs.txt
'Bos_taurus_cytochrome_c_oxidase_subunit_I_(mitochondrion)_complete_outputs.txt'
get_prots.py
Mammalia_COX1_complete_outputs.txt
Nematoda_COX1_complete_outputs.txt
```

You might want to add other files?

```
git add *.py
```

git commit -m "Scripts from the BioPython lecture"

```
[master bf575a2] Scripts from the BioPython lecture
1 file changed, 148 insertions(+)
create mode 100644 Lecture20/get_prots.py
```

git push AllLectures master

```
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 256 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 2.01 KiB | 137.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote: Internal Server Error
To https://github.com/B123456-2121/BPSM_LectureStuff.git
```

```
! [remote rejected] master -> master (Internal Server Error)
error: failed to push some refs to 'https://github.com/B123456-2121/BPSM_LectureStuff.git'
```

what did that mean!?

git log

```
bf575a2 (HEAD -> master) Scripts from the BioPython lecture
75d3a16 (AllLectures/master) Scripts from the python plots lecture
41e1ac8 Scripts from the python regex lecture
097800e Script from the python pandas lecture
1d2319e Scripts from the python dictionaries lecture
c49df45 Scripts from the python functions lecture
3b8bf41 My scripts from the python conditionals exercises Py04
8f58f4c Scripts from the python lists and loops lecture 3
e444ef8 Scripts from the second BPSM python lecture
6d6f5e7 My python scripts from the first BPSM python lecture
eae65c6 (Lecture07/master) edirect seqs related stuff added
eea4f5e (Lec_6_branch) Kept bash script output files from Lecture 06
fbe5cc0 Added run_this_script_v1 from BPSM Lecture06
ce2c15a nematode BLASTDB stuff added
19cdd2d Added the example_people_data.tsv file from Lecture03
137c3db Decided to keep blastoutput2.out
08d379b Decided to keep myinputfile.tsv
c9b5b18 First attempt at parsing BLAST data
bbf825c My first awk script, Lecture05
68f27fb There were conflicts with someotherfile.sh, kept all changes
83629cb (Version1.2) Final version of the someotherfile.sh shell script prior to merge
0b5425b (tag: v2.0) Updated contents of tar, coolscript
5d8aeea I am not sure cool script is actually any good
bbd36e7 Added my_cool_script
ea60357 Third version of the someotherfile.sh shell script
b6ae4a7 Updated version of tar
ce230e1 (tag: v1.2) Second version of the someotherfile.sh shell script
a5add7c Added the someotherfile.sh shell script
101b700 Additional motif files, and updated tar file
d21cb3d Adding intial tar file
59d1756 CodeFiles all done and tar zipped
6dd85a8 First file added
```

Have your say...

The "Postbox of Truth": is your chance to give anonymous "in course" feedback/comments. Your comments are completely anonymous and confidential, so say what you think, irrespective of whether the comments are positive or otherwise! You can use this multiple times too: I use it throughout the course to enable you to give context-relevant feedback/make suggestions.

Just click the image!



[Sources used](#)